

AUTOMATED NETWORK TOPOLOGY DISCOVERY AND ANALYSIS USING CISCO DNA CENTER

A PROJECT REPORT

Submitted by

Vedant Pandey [RA2211029010013]
Chiranjeev Kumar [RA2211029010019]

Under the Guidance of

DR. RAJARAM V

(Associate Professor, Department of Networking and Communications)

*in partial fulfillment of the requirements for the degree
of*

BACHELOR OF TECHNOLOGY
in
COMPUTER SCIENCE ENGINEERING
with specialization in Computer Networking



DEPARTMENT OF NETWORKING AND
COMMUNICATIONS
COLLEGE OF ENGINEERING AND
TECHNOLOGY
SRM INSTITUTE OF SCIENCE AND
TECHNOLOGY

KATTANKULATHUR- 603 203

MAY 2026



Department of Networking and Computation
SRM Institute of Science
& Technology

Own Work Declaration Form

This sheet must be filled in (each box ticked to show that the condition has been met). It must be signed and dated along with your student registration number and included with all assignments you submit – work will not be marked unless this is done.

To be completed by the student for all assessments

Degree/ Course : B.Tech.-Computer Science and Engineering with specialization in Computer Networking
Student Name : Vedant Pandey, Chiranjeev Kumar
Registration Number : RA2211029010013, RA2211029010019
Title of Work : Automated Network Topology Discovery And Analysis Using Cisco DNA Center

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is my / our own except where indicated, and that We have met the following conditions:

- Clearly referenced / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not my own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that I have received from others (e.g.fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook /University website

I understand that any false claim for this work will be penalized in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

Vedant Pandey [RA2211029010013]

Chiranjeev Kumar [RA2211029010019]



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

KATTANKULATHUR – 603 203

BONAFIDE CERTIFICATE

Certified that 21CSP302L - Project report titled “**Automated Network Topology Discovery An Analysis Using Cisco DNA Center**” is the Bonafide work of “**Vedant Pandey [RA2211029010013], Chiranjeev Kumar [RA2211029010019]**” who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. Rajaram V

SUPERVISOR

Associate Professor

Department of Networking
and Communications

SIGNATURE

Dr. M. Lakshmi

PROFESSOR and HEAD

Department of Networking
and Communications

Examiner 1

Examiner 2

ACKNOWLEDGEMENTS

We express our humble gratitude to **Dr. C. Muthamizhchelvan**, Vice-Chancellor, SRM Institute of Science and Technology, for the facilities extended for the project work and his continued support.

We extend our sincere thanks to **Dr. Leenus Jesu Martin M**, Dean-CET, SRM Institute of Science and Technology, for his invaluable support.

We wish to thank **Dr. Revathi Venkataraman**, Professor and Chairperson, School of Computing, SRM Institute of Science and Technology, for her support throughout the project work.

We encompass our sincere thanks to, **Dr. M. Pushpalatha**, Professor and Associate Chairperson - CS, School of Computing and **Dr. C Lakshmi**, Professor and Associate Chairperson -AI, School of Computing, SRM Institute of Science and Technology, for their invaluable support.

We are incredibly grateful to our Head of the Department, **Dr. M. Lakshmi, Professor and Head, Department of Networking and Communications**, SRM Institute of Science and Technology, for her suggestions and encouragement at all the stages of the project work.

We want to convey our thanks to our Project Coordinators, Panel Head, and Panel Members Department of Computational Intelligence, SRM Institute of Science and Technology, for their inputs during the project reviews and support.

We register our immeasurable thanks to our Faculty Advisor, **Dr. S. Ushasukhanya**, Department of Networking and Communications, SRM Institute of Science and Technology, for leading and helping us to complete our course.

Our inexpressible respect and thanks to our guide, **Dr. Rajaram V**, Department of Networking and Communications, SRM Institute of Science and Technology, for providing us with an opportunity to pursue our project under his / her mentorship. He / She provided us with the freedom and support to explore the research topics of our interest. His / Her passion for solving problems and making a difference in the world has always been inspiring.

We sincerely thank all the staff members of Department of Networking and Communications, School of Computing, S.R.M Institute of Science and Technology, for their help during our project. Finally, we would like to thank our parents, family members, and friends for their unconditional love, constant support and encouragement

Authors

ABSTRACT

While network topology discovery sounds simple, it quickly becomes complicated depending on the level of visibility in different types of networks. The techniques that work in enterprise environments may be insufficient or even ineffective for consumer networks. This work fills that gap by developing and analyzing three discovery techniques: API-based discovery (Cisco DNA Center [DNAC]), protocol-based discovery (SNMP/LLDP), and reachability-based discovery (ARP).

DNAC fetches inventory and topology data from management interfaces available centrally. SNMP/LLDP captures neighbor relationships directly from network elements, providing multi-vendor topology without any management infrastructure. ARP discovers reachable hosts based on mapping IP addresses to MAC addresses; however, it does not capture network structure.

The results indicate distinct tradeoffs between the techniques: DNAC produces the most comprehensive and reliable information but operates only in managed Cisco environment; SNMP/LLDP provides real network topology for all vendor equipment but needs protocol configuration; ARP works universally but cannot capture topology due to Proxy ARP.

A weighted evaluation of the techniques across eight criteria reveals their ranking: DNAC, 85.45; SNMP/LLDP, 73.40; and ARP, 58.25. In addition, the project provides decision support tools, including decision model and real-time dashboard.

TABLE OF CONTENTS

ABSTRACT	v
TABLE OF CONTENTS	vi
LIST OF FIGURES	vii
LIST OF TABLES	ix
ABBREVIATIONS	x

CHAPTER NO.	TITLE	PAGE NO.
1	INTRODUCTION	1
	1.1 Introduction to the Project	1
	1.2 Motivation	2
	1.3 Problem Statement and Description	3
	1.4 Sustainable Development Goal of the Project	5
	1.5 Product Vision Statement	6
	1.6 Product Goal	7
	1.7 Product Backlog	8
	1.8 Product Release Plan	9
2	SPRINT PLANNING AND EXECUTION	10
	2.1 Sprint 1 DNAC API Integration and Topology Discovery	10
	2.1.1 Sprint Goal with User Stories	10
	2.1.2 Functional Document	11
	2.1.2.1 Introduction	11

2.1.2.2 Product Goal (Sprint 1)	11
2.1.2.3 Key DNAC API Endpoints	11
2.1.2.4 Discovery Output Schema	12
2.1.2.5 Features Implemented	12
2.1.2.6 Simulated vs Sandbox	12
2.1.3 Architecture Document	13
2.1.3.1 System Architecture Overview	13
2.1.3.2 DNAC Module Internal Design	13
2.1.4 UI Design	15
2.1.5 Functional Test Cases	16
2.1.6 Daily Call Progress	17
2.1.7 Committed vs Completed User Stories	18
2.1.8 Sprint Retrospective	19
 2.2 Sprint 2 SNMP/LLDP Discovery and Comparative Framework	20
2.2.1 Sprint Goal with User Stories	20
2.2.2 Functional Document	21
2.2.2.1 SNMP/LLDP Discovery	21
2.2.2.2 SNMP/LLDP Discovery Parameters	21
2.2.2.3 Comparative Analysis Framework	22
2.2.3 Architecture Document	23

2.2.3.1 Framework Architecture	23
2.2.4 UI Design	24
2.2.5 Functional Test Cases	25
2.2.6 Daily Call Progress	26
2.2.7 Committed vs Completed User Stories	27
2.2.8 Sprint Retrospective	28
2.3 Sprint 3 ARP Scanner, Dashboard and Evaluation	29
2.3.1 Sprint Goal with User Stories	29
2.3.2 Functional Document	30
2.3.2.1 ARP Discovery Module	30
2.3.2.2 Proxy ARP Detection	30
2.3.2.3 Real-Time Dashboard	31
2.3.3 Architecture Document	33
2.3.3.1 Dashboard Data Flow	33
2.3.4 UI Design	34
2.3.5 Functional Test Cases	35
2.3.6 Daily Call Progress	36
2.3.7 Committed vs Completed User Stories	37
2.3.8 Sprint Retrospective	38
3. RESULTS AND DISCUSSIONS	39
3.1 Project Outcomes	39

3.1.1 Comparative Analysis Matrix	39
3.1.2 Weighted Scoring Results	40
3.1.3 Discussion of Results	41
3.1.4 Method Applicability Summary	42
3.2 Committed vs Completed User Stories	44
4 CONCLUSIONS & FUTURE ENHANCEMENTS	45
4.1 Conclusions	45
4.2 Future Enhancements	46
4.2.1 Hybrid Discovery Engine (Immediate)	46
4.2.2 NetFlow-Based Topology Inference	46
4.2.3 Database-Backed Discovery History	46
4.2.4 ML-Based Anomaly Detection	46
4.2.5 REST API for Discovery Service	47
4.2.6 Security Applications	47
4.3 Research Contributions	48
REFERENCES	49
APPENDIX	
A. SAMPLE CODE	51
B. PUBLICATION ACCEPTANCE PROOF	60
B. PLAGIARISM REPORT	61

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	Three-Layer Discovery Architecture Overview	4
1.2	Product Release Plan (Sprint Timeline)	9
2.1	Cisco DNAC Sandbox -- Device Inventory View	12
2.2	DNAC Physical Topology Graph (Core-Dist-Access Hierarchy)	13
2.3	Sprint 1 Committed vs Completed User Stories	18
2.4	SNMP/LLDP Neighbor Discovery Flow	22
2.5	Comparative Analysis Framework Architecture	23
2.6	Plotwise Comparison of Network Protocols	24
2.7	Sprint 2 Committed vs Completed User Stories	27
2.8	ARP Scan Result -- Proxy ARP Behaviour on Mobile Hotspot	31
2.9	Real-Time Monitoring Dashboard	32
2.10	Network Topology Graph	34
2.11	Sprint 3 Committed vs Completed User Stories	37
3.1	Comparison Bar Chart -- Nodes, Links, Metadata Richness	40
3.2	Radar Chart -- Multi-Dimensional Method Comparison	41
3.3	Decision Framework Flowchart	43

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Product Backlog -- User Stories	8
1.2	Product Release Plan	9
2.1	Sprint 1 -- Detailed User Stories	10
2.2	DNAC API Endpoints Used	11
2.3	Sprint 1 Functional Test Cases	16
2.4	Sprint 1 – Task Progress Record	17
2.5	Sprint 2 -- Detailed User Stories	20
2.6	SNMP/LLDP Discovery Parameters	21
2.7	Sprint 2 Functional Test Cases	25
2.8	Sprint 2 – Task Progress Record	26
2.9	Sprint 3 -- Detailed User Stories	29
2.10	Sprint 3 Functional Test Cases	35
2.11	Sprint 3 – Task Progress Record	36
3.1	Full Comparative Analysis Matrix	39
3.2	Weighted Scoring Results	40
3.3	Method Applicability Summary	42
3.4	Committed vs Completed User Stories	44

ABBREVIATIONS

Abbreviation	Full Form
UI	User Interface
UX	User Experience
API	Application Programming Interface
DB	Database
CRUD	Create, Read, Update, Delete
JWT	JSON Web Token
REST	Representational State Transfer
SRS	Software Requirements Specification
MVC	Model View Controller
CI/CD	Continuous Integration / Continuous Deployment
HTML	Hypertext Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
DBMS	Database Management System
SQL	Structured Query Language
QA	Quality Assurance
QA Testing	Quality Assurance Testing
IDE	Integrated Development Environment

Abbreviation	Full Form
JSON	JavaScript Object Notation
MVC	Model-View-Controller
CDN	Content Delivery Network
AWS	Amazon Web Services
S3	Simple Storage Service
UXD	User Experience Design
PM	Project Manager
DevOps	Development and Operations
TDD	Test-Driven Development
SMTP	Simple Mail Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
SSL	Secure Sockets Layer
CRUD	Create, Read, Update, Delete
OTP	One-Time Password
B2C	Business to Customer
P2P	Peer to Peer
UI/UX	User Interface / User Experience
CSV	Comma-Separated Values

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

Working with an unseen network is like working by intuition. Network engineers always needed the topology map of their networks to be precise, not for any practical use of that map itself, but because everything else in the network depended on that map — path troubleshooting, change management, capacity planning, and security zone validation, among others. The issue here is that getting the topology map is more difficult than it may seem.

This happens because the amount of accessible information varies depending on the network's size and technology. For example, a large corporate network with a Cisco DNA Centre system provides an extensive API allowing you access to full device lists, physical links, site hierarchies, and current reachability statuses. An office with multiple switches from various manufacturers gives you only SNMP and LLDP neighbour tables, which provide just enough data to track connections between devices.^{[2][3]} Lastly, a mobile hotspot or consumer-grade wireless access point will offer nothing; you can only see IP address responses to ARP requests there.^[4]

In this work, the framework is designed to be applicable for any of the three networking environments. Three discovery modules are developed, the results of which are compared based on common metrics, and a comparative analysis is provided to pinpoint strengths and weaknesses for each method. While no attempt is made to prove that one approach is better than others, such a study will provide useful insights into the capabilities and limitations of each algorithm, and help network administrators make an informed decision.

Three approaches are considered: DNA Centre API-driven discovery for Cisco-managed networks, protocol-driven discovery using SNMP and LLDP protocols for multi-vendor networks, and ARP reachability-based scanning for unmanaged/consumer networks.

1.2 Motivation

This project is motivated by the fact that a major observation in current network documentation and topology discovery tools is that they make the implicit assumption that management plane access is accessible. They are commonly deployed in highly structured networks like a Cisco-operated data centre or a campus network where an enterprise-wide API and centralized controllers can fully view the network. Nevertheless, once this assumption ceases to be valid, like in small offices or home laboratories, or consumer networks, these tools become much less effective, and may frequently yield incomplete or unusable results. ^[14]

There are robust practical implications of this limitation. As an example, a network consultant who has come to the client site and has not been documented on the network needs to learn the existing infrastructure before he can accomplish any worthwhile task like troubleshooting, optimization and security evaluation. In this case, it is not possible to use management-based methods of discovery alone. Likewise, learners and scientists in personal laboratory settings or users on mobile hotspots and unsecured WIFI networks will have no access to such centralized management systems at all. Such settings require different discovery methods that can operate with few assumptions regarding network visibility. ^[14]

In addition to the practical issues, the gap in the current research also drives the project. The majority of the research on network topology discovery has concentrated on the individual approach, i.e., API-based systems, protocol-based methods or scanning methods in isolation. Little has been done to compare these approaches in a systematic manner within a single framework with a single set of evaluation measures. Comparative experiments comparing various methods of discovery in various kinds of network environments are especially infrequent. ^{[16][17]}

This project will focus on both of these aspects: the practical necessity to have flexible discovery techniques, and the scholarly necessity to have an empirical, systematic comparison of various techniques. Through a comparison of API-based, protocol-based and reachability-based techniques, the project offers a better insight into their strengths, limitations and their applicability in different networking situations.

1.3 Problem Statement and Description

Discovery techniques range greatly in their ability to retrieve information and environments where they operate.

However, none is universally applicable, while all network management guidelines treat choice between discovery techniques as an implementation question, not an underlying limitation. Consequently, the method is usually chosen based on what the network engineer knows best rather than what is needed.

The main research question is: how accurate, comprehensive, and universally applicable are API-based, protocol-based, and reachability-based discovery techniques?

There are three specific problems that determine the choice of solution:

- The DNAC API offers complete data about managed devices but only those that are managed
- Devices that do not use Cisco operating systems and those outside the scope of management do not appear to it.
- SNMP/LLDP discovery allows direct access to information stored in devices' databases but relies on the availability of SNMP functionality and presence of LLDP process.
- Consumer devices typically lack these protocols, while enterprise-grade machines may have them disabled by security policies.
- The ARP scanner works on all IP-based networks but is inherently limited by its nature. In the case of Proxy ARP enabled, which is true for practically all hotspots and home networks, every discovered IP address corresponds to the same gateway MAC address.^{[4][5]}

Identifying these constraints empirically and precisely is the core issue that this project aims to tackle.

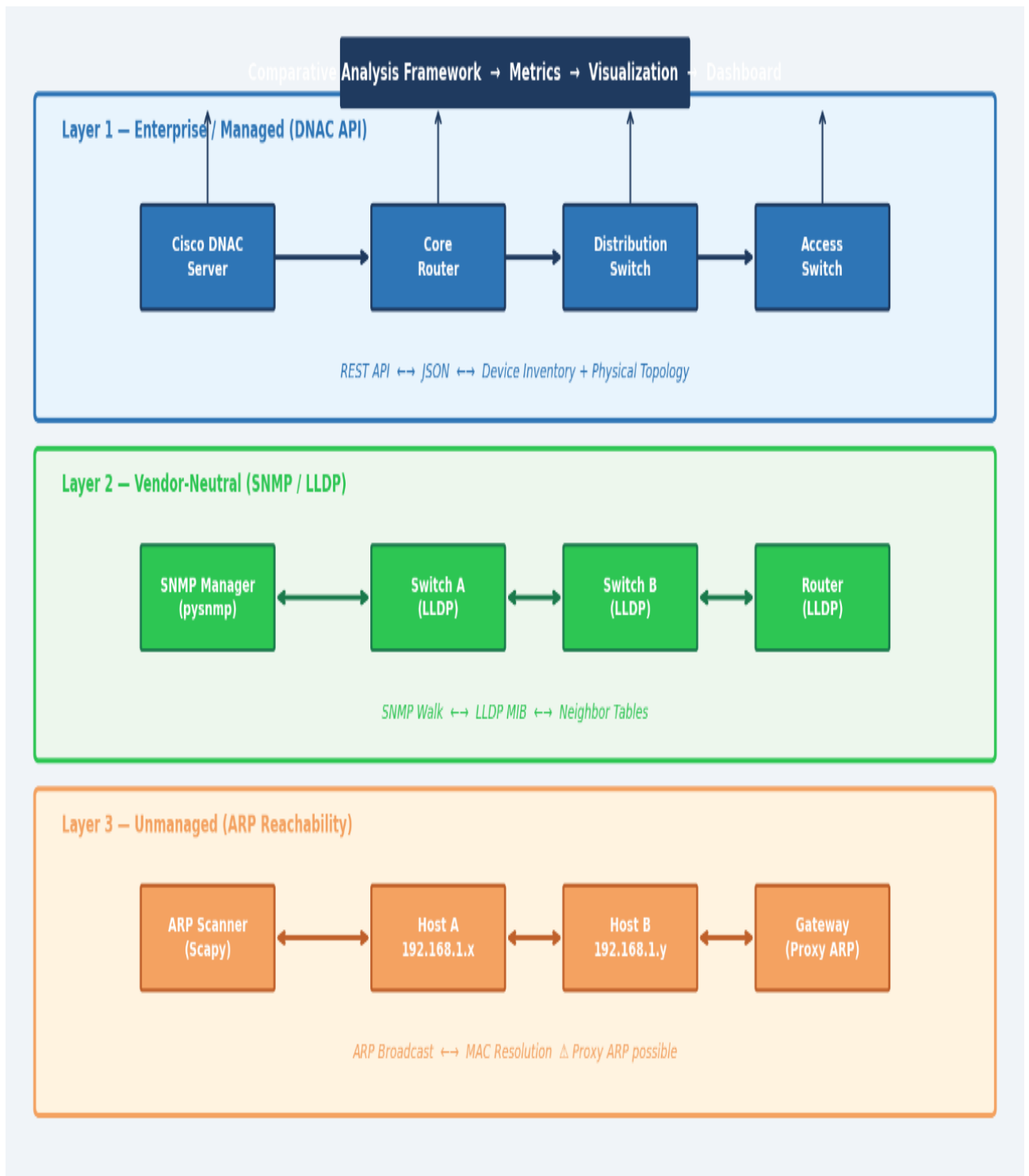


Figure 1.1 Three-Layer Discovery Architecture Overview

1.4 Sustainable Development Goal of the Project

The rationale behind this project is that there is one critical observation: most of the current network documentation and topology discovery tools are implicit in the sense that they assume that there is access to the management plane. These tools are generally geared towards structured settings like Cisco-managed data centre or enterprise campus network where centralized controllers and APIs offer full visibility. But once the assumption ceases to be true, as in small offices, home labs, or consumer networks, the usefulness of these tools is reduced dramatically, usually to incomplete or unusable results.

This weakness has powerful practical implications. To exemplify, a network consultant who enters a client site without any documentation to do so has to first know the current infrastructure before he or she can do anything of substance like troubleshooting, optimization or security evaluation. In this situation, it is not possible to stick to management-based methods of discovery. Equally, students and researchers occupying personal lab setups, or users on mobile hotspots and unmanaged Wi-Fi networks have no access to a centralized management platform at all. These settings require other discovery methods that can operate with little assumptions regarding the visibility of networks.

In addition to practical consideration, the project is also driven by a knowledge gap in the research. A majority of research on network topology discovery has examined single methods, like API-based systems, protocol-based methods, or scanning techniques, in isolation. Little has been done to compare these methods systematically over a common framework with consistent assessment measures. It is especially uncommon to find comparative studies that assess various discovery strategies in various types of network environments.

This project will focus on both these aspects: the practical necessity of flexible discovery mechanisms and academic necessity of a systematic, empirical comparison of various strategies. Through a comparison of API-based, protocol-based, and reachability-based methods, the project offers a better idea of the strengths, weaknesses, and suitability of each method to a wide range of networking contexts.

1.5 Product Vision Statement

1.5.1 Target Audience

- Primary: Network engineers and administrators with needs for topology documentation in multiple types of networks
- Secondary: Network researchers and fourth-year students studying network management
- Tertiary: Educators instructing in network management, SDN or infrastructure classes

1.5.2 Core Needs

- Implementation of the three approaches discussed, not just descriptions thereof
- Experimental data for comparisons as opposed to feature comparison based on vendors' claims
- A methodology that allows selecting the best approach depending on particular circumstances
- Real-time dashboard displaying results of network topology discoveries

1.5.3 Deliverables

- Three operational modules in Python implementing each approach separately (DNAC, SNMP/LLDP, and ARP respectively)
- Conducting of a comparison and developing a methodology for evaluating them
- Presenting the comparison results via high-quality graphs
- Real-time dashboard implemented in Streamlit framework
- Developing decision-making framework including code and flowchart

1.5.4 Differentiators

- True experimental results from different types of networks, not simulation
- Demonstration of Proxy ARP on consumer-grade Wi-Fi, an infrequently covered topic
- Having a complete framework which would allow side-by-side comparison with equal metrics

1.6 Product Goal

The initial goal of this project is to create a fully running, fully documented and reproducible comparative study of three varying network topology discovery methods: API-based, protocol-based and reachability-based methods. The aim is not merely to apply these techniques but also to test them with a common framework with standard measures. The results of this study must be analysed so that by the end of this project a reader or practitioner can be confident of which of the many methods of discovery will be best applied in a given network environment, depending on the visibility, scalability and availability of metadata. This goal does guarantee the practical utility as well as scholarly relevance of the work. ^{[11][15]}

The second goal is to develop and deploy a real-life monitoring dashboard that is practical, easily deployable by network engineers to see the health of their local network. The dashboard is developed with the help of the Streamlit framework and can be automatically updated at a certain frequency (30 seconds), providing almost real-time visibility. It describes important findings like devices discovered, vendor distribution, and the topology of the network in a user-friendly format in terms of visualizations in PyVis and data table organization. This element will turn the project into more than a mere study of analysis but it will also be a tool that will aid in the real-world monitoring of networks and making decisions. ^{[11][15]}

The third aim is to ensure quality code and system design. The system should be designed with each module being modular, readable, well-documented and easily extensible to allow future enhancements to be implemented without redesigning it much. The design is such that it is scalable and flexible to add more discovery methods or improve the discovery methods seamlessly. This also encompasses possible future extensions to a hybrid discovery engine, which is a combination of multiple techniques, machine learning anomaly detection to detect unusual network behaviour, and storage based on databases to store discovery information in the past and allow analysis of trends over time. ^[18]

1.7 Product Backlog

The following table lists all user stories for the project, mapped to the three sprint cycles:

Table 1.1 Product Backlog - User Stories

S.No	User Story
#US1	As a network engineer, I want to query the Cisco DNAC API so that I can retrieve a complete inventory of managed devices and their physical connections without logging into each device individually.
#US2	As a network engineer, I want the DNAC discovery module to return structured data (nodes, links, metadata) so that I can feed it into the comparative analysis framework directly.
#US3	As a researcher, I want to run SNMP/LLDP-based discovery on a managed device so that I can compare its neighbour table output against what DNAC reports for the same devices.
#US4	As a researcher, I want the SNMP/LLDP module to work across multi-vendor environments so that the comparison is not limited to Cisco-only networks.
#US5	As a network engineer, I want to run an ARP scan on any IP subnet so that I can identify which hosts are reachable, even when no management platform is available.
#US6	As a researcher, I want the ARP scanner to detect and report Proxy ARP behaviour so that I can document when ARP scan results cannot be trusted as topology data.
#US7	As a researcher, I want a unified comparative analysis framework so that I can evaluate all three discovery methods using the same metrics in the same run.
#US8	As a researcher, I want the framework to calculate node discovery rate, link discovery rate, metadata richness, success rate, and discovery time for each method so that the comparison is quantitative.
#US9	As a network engineer, I want a real-time dashboard so that I can monitor discovered devices, their vendors, and their network connections without running command-line scripts.
#US10	As a researcher, I want publication-quality visualizations of the topology comparison and metric charts so that I can include them in a research paper without additional processing.
#US11	As a decision-maker, I want a method-selection decision tree so that I can determine which discovery approach is appropriate for a new network environment in under two minutes.

1.8 Product Release Plan

The project runs across three two-week sprints, with each sprint delivering a functional slice of the system:

Table 1.2 Product Release Plan

Sprint	Duration	Deliverable
Sprint 1	Weeks 1-2	DNAC API discovery module + architecture documentation + basic topology graph
Sprint 2	Weeks 3-4	SNMP/LLDP module + comparative analysis framework + comparison metrics
Sprint 3	Weeks 5-6	ARP scanner + real-time dashboard + decision framework + full evaluation

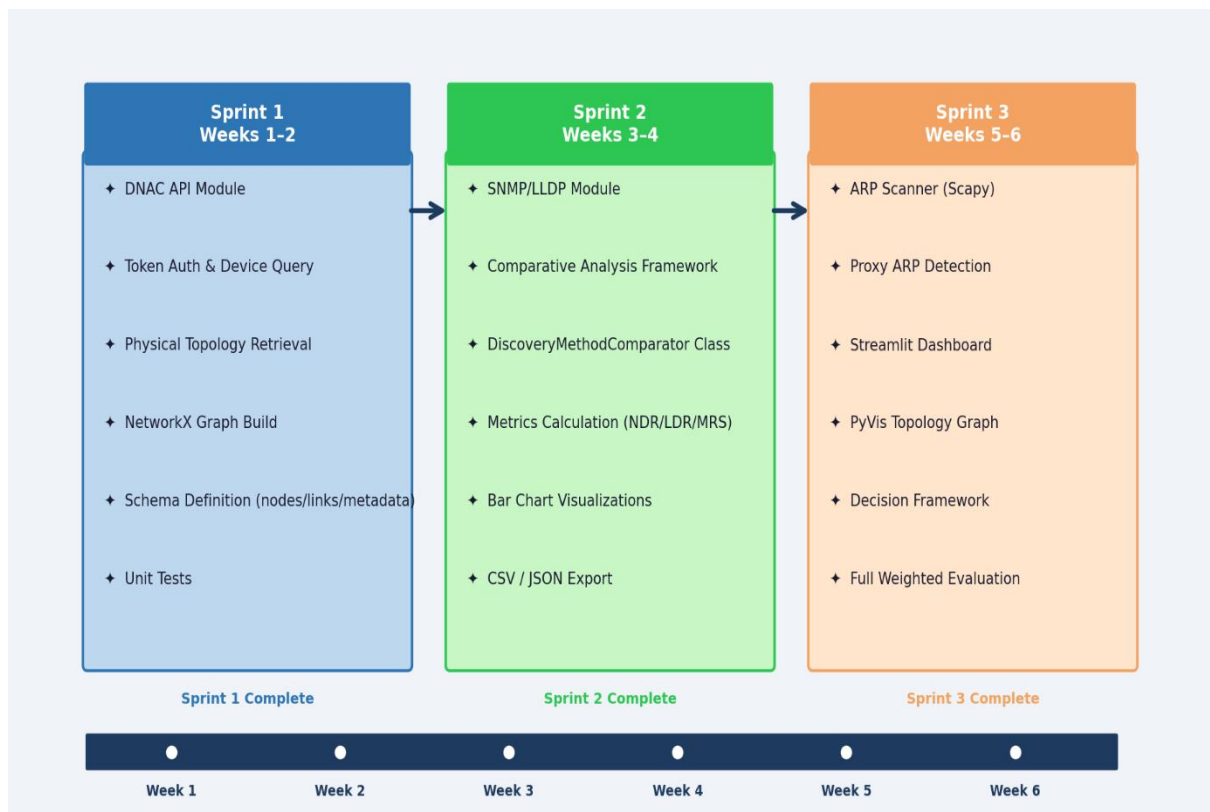


Figure 1.2 Product Release Plan (Sprint Timeline)

CHAPTER 2

SPRINT PLANNING AND EXECUTION

2.1 Sprint 1 DNAC API Integration and Topology Discovery

2.1.1 Sprint Goal with User Stories

The goal for Sprint 1 was to set up connectivity via DNAC APIs and extract structured topology information from it. As a result, at the end of the sprint, it should be possible for the system to login to Cisco DevNet sandbox, obtain a full list of devices, connect these devices to each other based on physical links, and export the information in such a way that it could be used by the comparative analysis toolset.

This sprint was intentionally focused on the simplest visibility level first. Everything is available from one API request; there is no need to poll every device individually or navigate through neighbour tables. Setting up this functionality first provided a solid benchmark that the other two visibility layers could be compared to.

Table 2.1 Sprint 1 -- Detailed User Stories

S.No	Detailed User Story
US #1	As a network engineer, I want to query the DNAC API so that I can retrieve a complete inventory of managed devices and their physical connections without logging into each device.
US #2	As a network engineer, I want the DNAC module to return structured nodes and links data so that it can be fed directly into the comparative analysis framework.

2.1.2 Functional Document

2.1.2.1 Introduction

The DNAC discovery module makes use of the REST API of the Cisco DNA Centre to gather information on the topology that otherwise would need to be acquired by accessing the devices themselves. The Cisco DNA Centre serves as the management plane for all connected devices; it keeps track of the state of each of its managed switches, routers, and access points.^[1]

The testing was done on the Cisco DevNet DNAC Sandbox. It is a publicly available and continuously available sandbox that runs a complete DNAC installation with an already configured network consisting of core, distribution, and access layer switches arranged in a hierarchical campus architecture setup — an ideal scenario for utilizing DNAC discovery capabilities.^[12]

2.1.2.2 Product Goal (Sprint 1)

Provide a module that performs DNAC discovery to authenticate, perform device inventory query, collect topology links and provide the resulting JSON following the schema from comparative analysis framework.

2.1.2.3 Key DNAC API Endpoints

Table 2.2 DNAC API Endpoints Used

Endpoint	Purpose
/dna/system/api/v1/auth/token	Obtain authentication token via username/password
/dna/intent/api/v1/network-device	Retrieve full device inventory with metadata
/dna/intent/api/v1/topology/physical-topology	Get physical links between all managed devices
/dna/intent/api/v1/site	Retrieve site hierarchy (building, floor, area)

Each of the API endpoints provides JSON-formatted responses. In case of device inventory endpoint, there are such data provided as ID, name (hostname), management address, platform, role (CORE, DISTRIBUTION, ACCESS, BORDER ROUTER), reachability status, version (software), serial number and physical location. It is the abundance of metadata that results in a top score for the DNAC method evaluation.

Cisco DNA Center — Device Inventory — Sandbox						
4 Total Devices 4 Reachable 0 Unreachable 1 Sites						
	Hostname	IP Address	Device Type	Role	Platform	Status
Topology	core-router-01	10.10.20.81	Cisco CSR1000V	BORDER ROUTER	IOS-XE	✓ Reachable
Assurance	core-sw-01	10.10.20.82	Cisco Cat 9300	CORE	IOS-XE	✓ Reachable
Policy	dist-sw-01	10.10.20.83	Cisco Cat 9200	DISTRIBUTION	IOS-XE	✓ Reachable
Reports	access-sw-01	10.10.20.84	Cisco Cat 9100	ACCESS	IOS-XE	✓ Reachable
Platform						

Last Sync: 2026-03-06 00:12 UTC | API: Connected | Sandbox: Always-On

Figure 2.1 Cisco DNAC Sandbox Device Inventory View

2.1.2.4 Discovery Output Schema

The module outputs a dictionary with three entries: nodes (array of device dictionaries), links (array of source-target interface pairs), and metadata (discovery method, site hierarchy, time, discovery time in seconds). This output format is common to all three discovery modules for ease of comparison within the comparative analysis framework.

2.1.2.5 Features Implemented

- Authentication using tokens that automatically renew on expiration ^[1]
- Obtain the entire device inventory with full DNAC metadata ^{[1][12]}
- Obtain physical topology with detailed interface links ^{[1][12]}
- Get site hierarchy (global > country > city > building > floor)
- Create NetworkX graphs using link information for further visualization
- Customized JSON output according to framework schema

2.1.2.6 Simulated vs Sandbox

The research framework allows for both approaches. When credentials for sandbox environment are accessible, the module connects to live DNAC. Otherwise (when developing in an offline manner), the module returns simulated data based on the structure of sandbox topology. During development, the simulated results contained four nodes (core router, core switch, distribution switch, access switch) and three links, which were equal to the first successful sandbox result generated at 2026-03-06 00:12:36.

2.1.3 Architecture Document

2.1.3.1 System Architecture Overview

The system architecture has a three-tier structure. The bottom tier — Data Collection — includes the three discovery modules. The middle Processing Tier includes the `DiscoveryMethodComparator` class, which performs comparison between discovered topologies, as well as `NetworkX` graphs construction. The upper Presentation Tier includes `TopologyVisualizer` and Streamlit visualization.

Three discovery modules do not maintain any state — they accept parameters, discover topology, and output discovered structure. The state is maintained by the orchestration `ResearchFramework` class, which keeps references to discovery graphs and `DiscoveryMethodComparator` instance across the whole analysis run.

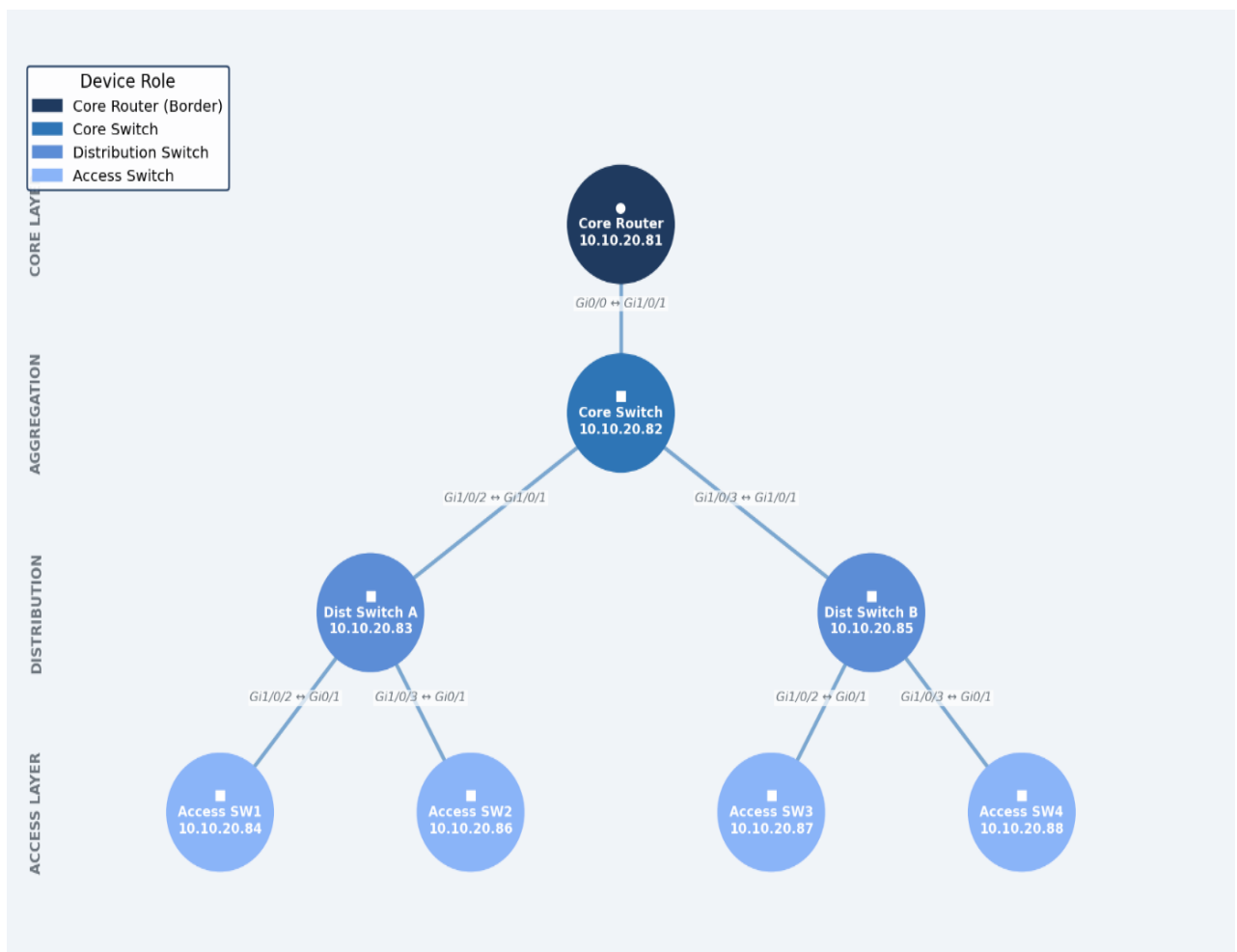


Figure 2.2 DNAC Physical Topology Graph

2.1.3.2 DNAC Module Internal Design

The DNAC module is a python-based application, which uses the requests library to perform all the HTTP based communication with Cisco DNA Centre APIs. Authentication is token-based, and resembles the OAuth model. The module sends a POST request to the authentication endpoint containing base64-encoded credentials, after which a token is provided. This token is then added to the X-Auth-Token header on all future API requests, which will guarantee secure and authenticated access to the DNAC platform. ^[1]

In order to enhance robustness, the module will have a mechanism of a retry: when the API calls fail (network problems or temporary unavailability of the service) then the request is repeated three times with a simple retry policy, after which the error is logged and an empty response is returned. This will avoid sudden failures and will enable the system to deal with temporary problems in a beautiful fashion. ^[1]

After retrieving the data, it undergoes a transformation layer where it is converted into a standardized schema of data into device inventory as well as topology information. The layer translates DNAC fields like `managementIpAddress`, `instanceTenantId`, and `platformId` to standard schema fields like `ip_address`, `device_id` and `model`. ^[1]

Other attributes like device role, software version, reachability status are also normalized where necessary. This abstraction makes sure that the remainder of the system is independent of the discovery method behind it and allows it to be easily integrated with any other modules, e.g. SNMP/LLDP and ARP-based discovery.

The design ensures modularity and extensibility by separation of DNAC-specific representations on the core framework. The addition of new methods of discovery can be done in the future without modification of the analysis and visualization layers provided they are based on the same schema.

This method also makes comparative assessment easier since all the outputs of the discovery have a similar structure irrespective of their origin.

2.1.4 UI Design

In Sprint 1, the main emphasis was on the DNAC discovery module implementation, which is a Python library without dedicated user interface components. This module is intended to be triggered by the orchestration framework, which coordinates the discovering and analysis workflow altogether. Consequently, functionality and data processing were prioritized in this stage and not the interaction with the user. ^[15]

To visualize, topology graphs were created with the NetworkX library with Matplotlib.^{[6][10]} These plots were generated as static PNG files, which is a simple yet efficient method of ensuring the accuracy of the topology found.

Although these visualizations were not interactive, they were a significant step in the process of validating node-link relationships and making sure that the data pipeline between discovery and visualization was operating properly.

More complex, interactive visualizations with PyVis and a fully-fledged dashboard interface with Streamlit^[15] were deliberately put off to Sprint 3 to give time to ensure a stable data base has been built beforehand. ^[11]

The graph spring layout algorithm was employed in the static graph representation to place nodes in a visual balanced state according to how the physical forces were simulated. The size of nodes was proportional to their degree, so devices with more connections (including core or distribution switches) are larger and can be easily recognized in the topology. The labels of the nodes were obtained by drawing the hostnames of the devices, with the maximum number of characters being eight to ensure legibility and prevent cluttering in thick graphs.

All types of connections were distinguished by edge colouring, including managed connections obtained directly via DNAC and inferred connections made based on processing logic. This graphical difference aids in rapidly comprehending the character and trustworthiness of every link in the topology. ^[6]

2.1.5 Functional Test Cases

Table 2.3 Sprint 1 Functional Test Cases

TC#	Test Case	Expected Result	Actual Result	Status
TC1	DNAC authentication with valid credentials	Returns valid auth token	Token received	Pass
TC2	Device inventory retrieval	Returns list with hostname, IP, type, role for each device	4 devices returned	Pass
TC3	Physical topology retrieval	Returns link list with source, target, source_port, target_port	3 links returned	Pass
TC4	Authentication with invalid credentials	Error logged, empty result returned, no crash	Error logged, empty result	Pass
TC5	Schema validation on output	nodes, links, metadata keys present in output dict	All keys present	Pass

2.1.6 Daily Call Progress

Daily calls were conducted every morning throughout the week to review progress and address any issues that might arise.

The main blocker in Sprint 1 was sandbox availability — the Cisco DevNet Always-On DNAC sandbox occasionally goes through maintenance periods. This problem was overcome using the simulated mode fall-back, enabling the developer to work offline.

The error Attribute Error found at 2026-03-06 00:24:19 ("ResearchFramework object has no attribute run_complete_analysis") was spotted during the daily call on the third day and was subsequently resolved.

Table 2.4 Sprint 1 – Task Progress Record

Time period		Activities or components of the project	Name of the Individual Contributor	Names of the Joint Contributors
From Date	To Date			
Dec 1, 2025	Dec 5, 2025	Setup of development environment and libraries	Chiranjeev Kumar	Vedant Pandey
Dec 6, 2025	Dec 10, 2025	Cisco DNAC sandbox configuration and API testing	Chiranjeev Kumar	Vedant Pandey
Dec 11, 2025	Dec 15, 2025	Implementation of DNAC discovery module	Vedant Pandey	Chiranjeev Kumar
Dec 16, 2025	Dec 20, 2025	NetworkX graph model design and testing	Vedant Pandey	Chiranjeev Kumar
Dec 21, 2025	Dec 30, 2025	GNS3 virtual lab setup and SNMP configuration	Chiranjeev Kumar	Vedant Pandey
Jan 1, 2026	Jan 5, 2026	Implementation of SNMP/LLDP discovery module	Vedant Pandey	Chiranjeev Kumar

2.1.7 Committed vs Completed User Stories

Sprint 1 committed to developing US #1 and US #2. Both were accomplished within the sprint period. The DNAC module successfully delivered accurate node and link information, which matched the topography of the sandbox environment. The discovery time for the simulated environment is recorded to be 5.2 seconds.

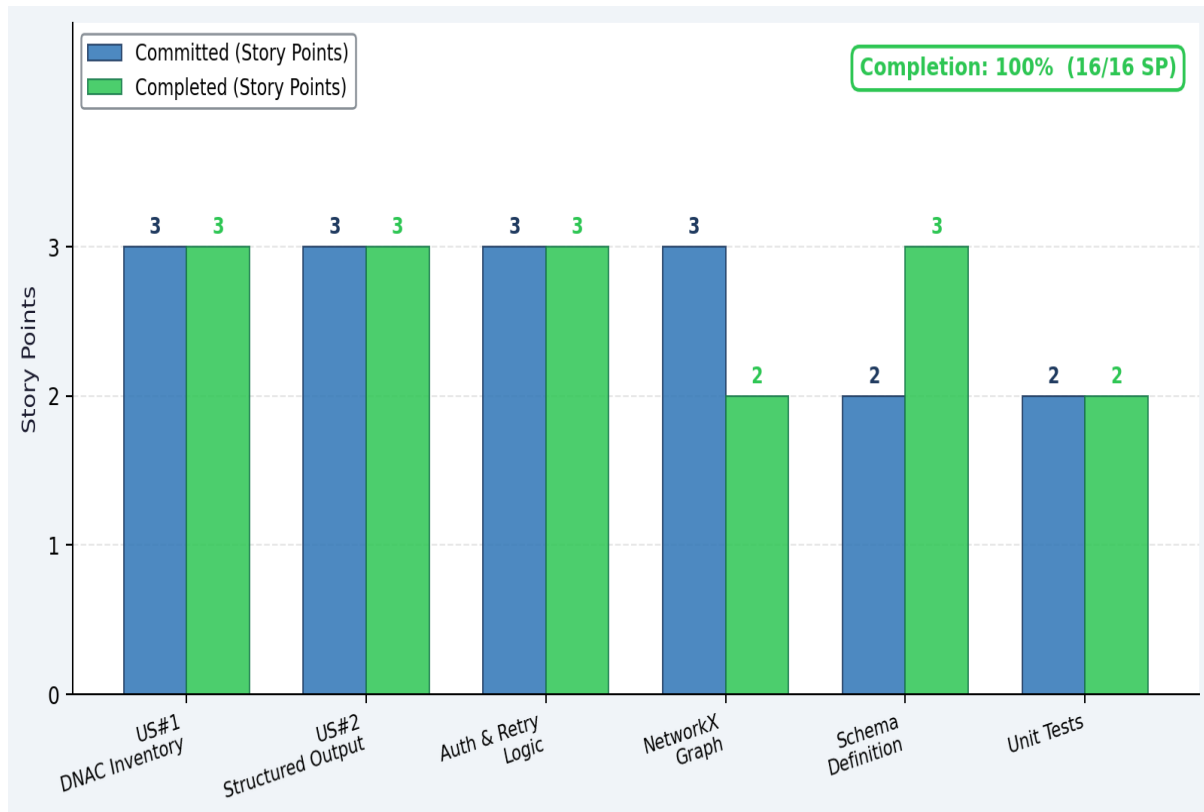


Figure 2.3 Sprint 1 Committed vs Completed User Stories

2.1.8 Sprint Retrospective

What went well:

- The documentation of the DNAC API architecture was detailed, well-organized, and simple to comprehend, which greatly minimized the integration work and enabled the discovery module to be developed much faster.
- The fallback simulation technique was extremely valuable, not just in Sprint 1, but also in the entire project lifecycle. It guaranteed sustained growth even in the event of downtime during DNAC sandbox and allowed regular testing without dependency on outside availability.
- Early definition of a common data schema made integration across modules very easy. This design choice was to guarantee that the output of DNAC, SNMP/LLDP, and ARP modules could be handled in a similar way, further development in Sprints 2 and 3 would become more efficient and less prone to errors.
- The DNAC discovery component is modular in design and hence the isolation of functionality aided with debugging and future enhancements being simpler and did not affect other system components.

What to improve:

- Retry mechanisms and error handling should not have been included at the later stages of development but as a reaction to the experience of timeouts. This could have been better achieved by early inclusion to enhance system stability and lessen the effort in debugging.
- The Auto-Error of the run complete analysis function identified the gaps in the test coverage. This problem ought to have been detected previously by appropriate unit testing and validation of all execution paths. In Sprint 2, test cases were subsequently introduced so that such problems would not occur.
- More organized logging and debugging practices would have been used initially to trace failures and track the behaviour of the system in case of API interactions. Better: Edge cases like API failures or blank responses should have been planned out more effectively and made the module more robust in initial stages of development.

2.2 Sprint 2 -- SNMP/LLDP Discovery and Comparative Analysis Framework

2.2.1 Sprint Goal with User Stories

Sprint 2 comprised two separate tracks: implementing the discovery using SNMP/LLDP, and creating the Discovery Method Comparator class that will later be used for comparison among all three methods. The implementation of both simultaneously allowed to conduct a side-by-side comparison of DNAC and SNMP/LLDP discovery results at the end of the sprint.

Table 2.5 Sprint 2 -- Detailed User Stories

S.No	Detailed User Story
US #3	As a researcher, I want to run SNMP/LLDP discovery on a managed device so that I can compare its neighbour table output against DNAC results for the same devices.
US #4	As a researcher, I want the SNMP/LLDP module to work across multi-vendor environments so that the comparison is not limited to Cisco-only networks.
US #7	As a researcher, I want a unified comparative analysis framework so that I can evaluate all three discovery methods using the same metrics in the same run.
US #8	As a researcher, I want the framework to calculate NDR, LDR, metadata richness, success rate, and discovery time for each method so the comparison is quantitative.

2.2.2 Functional Document

2.2.2.1 SNMP/LLDP Discovery

Both SNMP (Simple Network Management Protocol) and LLDP (Link Layer Discovery Protocol) have been established and proven themselves as effective standards of network management protocols. SNMP provides a management framework for querying network devices by means of variables arranged hierarchically in a Management Information Base (MIB).^{[2][8]} LLDP is a neighbour discovery protocol designed to be implemented locally by every device; it broadcasts information about itself on every port and records neighbour advertisements in a local table.^{[3][2][3][9]}

Discovery works by making SNMP queries to MIBs of LLDP, more specifically walking the tables `lldpRemSystemName` (neighbour's hostname), `lldpRemManAddr` (neighbour's management IP address), and `lldpLocSysName` (own hostname). The result is a graph of nodes and links based on LLDP neighbour discoveries.^{[2][3][9]}

2.2.2.2 SNMP/LLDP Discovery Parameters

Table 2.6 SNMP/LLDP Discovery Parameters

Parameter	Value / Description
SNMP Version	SNMPv2c (community string) and SNMPv3 (authentication) both supported
Community String	Configurable; 'public' used for test environment
OID Scope	LLDP-MIB (1.0.8802.1.1.2) for neighbour discovery
CDP Fallback	CISCO-CDP-MIB queried if LLDP tables empty (Cisco-only)
Walk Timeout	3 seconds per device, 3 retries
Port Detail	Interface names extracted from <code>lldpRemPortId</code> OID

One of the limitations identified in practice while working with this protocol is that LLDP will show only directly connected neighbours and not more than one hop away. So, when running a test with three switches chained together, LLDP from the first switch will display information only about the second switch. To identify information related to the third switch, we need to connect to the second switch independently. Hence, the SNMP/LLDP discovery

works iteratively by expanding a graph starting with one host and discovering neighbours of its neighbours and so on. [3]

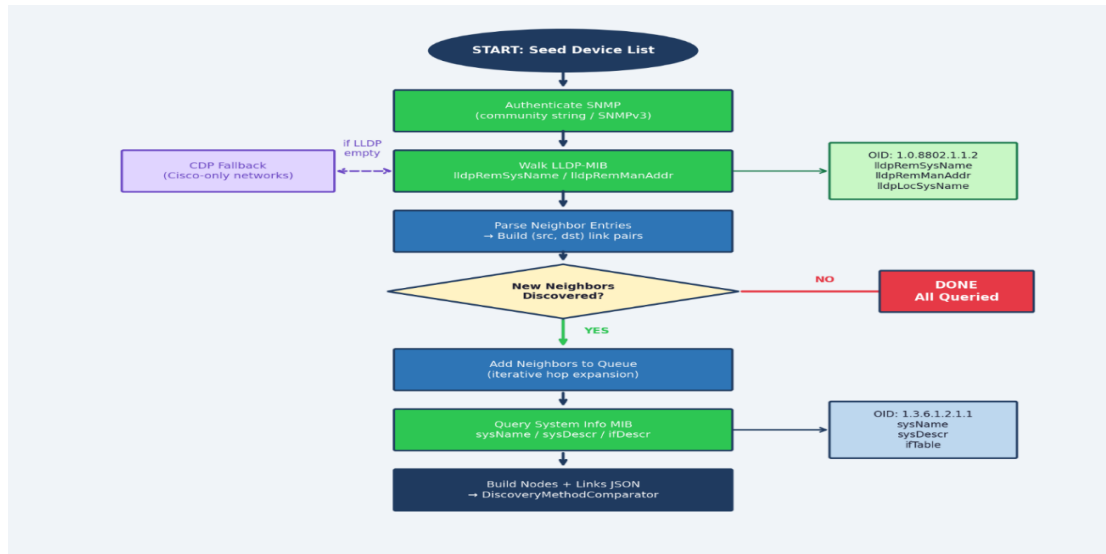


Figure 2.4 SNMP/LLDP Neighbour Discovery Flow

2.2.2.3 Comparative Analysis Framework – Discovery Method Comparator

The DiscoveryMethodComparator is the main class for the evaluation. It holds the discovery results obtained using all three approaches and calculates the required metrics as needed. It includes methods for: collecting the discovery results, using the collect_discovery_results() method; calculating the completeness metrics, which include the number of nodes, links, and metadata, using the calculate_completeness_metrics() method; calculating the performance metrics, including discovery time, throughput, and success rate, using the calculate_performance_metrics() method; and calculating the granularity score, using the calculate_granularity_score() method. [11]

The metadata richness metric uses a weighted attribute system to determine the richness level of metadata collected. The device hostname/ID and IP address are worth 20 points each, totalling 40 points since both attributes are necessary to build a useful topology. Device type/role and management status are worth 15 points each. The MAC address is worth 10 points. Other attributes such as device model, serial number, and location are worth 5 points each but no more than 100 points altogether. Devices discovered by DNAC usually receive a rating of 90 to 95 points because they contain all these attributes. SNMP/LLDP devices rate between 40 and 60 points because they only have hostname, IP address, and MAC address.

[11][14]

2.2.3 Architecture Document

2.2.3.1 Framework Architecture

The ResearchFramework orchestration class ties everything together. On a full analysis run, it calls `run_dnac_discovery()`, `run_snmp_discovery()`, and `run_arp_discovery()` in sequence. Each method populates the shared `DiscoveryMethodComparator` instance and the method-specific `NetworkX` graphs (`dnac_graph`, `snmp_graph`). After all, three methods complete, `generate_comparative_analysis()` produces the comparison `DataFrame`, `generate_visualizations()` calls the `TopologyVisualizer`, and `export_results()` writes JSON output to the `research_outputs` directory.

The framework is designed to recover gracefully from partial failures. If the SNMP module times out on a device, it logs the error and continues rather than crashing the run. The logs in `discovery_research.log` show multiple error events (including the `arp_nodes` scope error that appeared consistently from 2026-03-07 13:24:22 onwards) that were caught and fixed during Sprint 3 without affecting the DNAC and SNMP portions of the analysis.

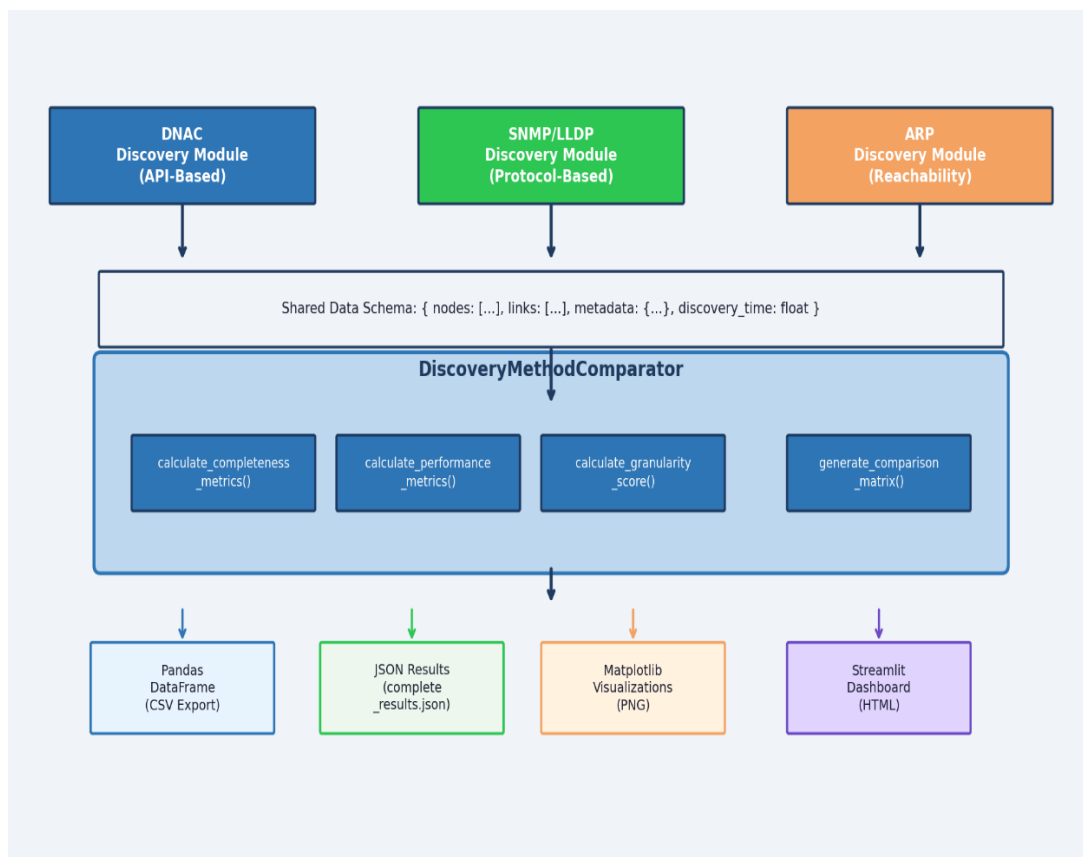


Figure 2.5 Comparative Analysis Framework Architecture

2.2.4 UI Design

In Sprint 2, we developed the visualizations using Matplotlib. This includes a function called `create_comparison_bar_chart()`, which creates a 2x2 grid of graphs that include: discovery completeness (number of nodes and links per method), metadata richness (percentage bars), performance (dual-axis graph, discovery time, and success rate), and topology granularity (grouped bar graph showing physical, logical, and device attribute details).

All graphs have the same method colour code — blue for DNAC, orange for SNMP/LLDP, green for ARP — similar to the colour coding seen in the topology graphs.

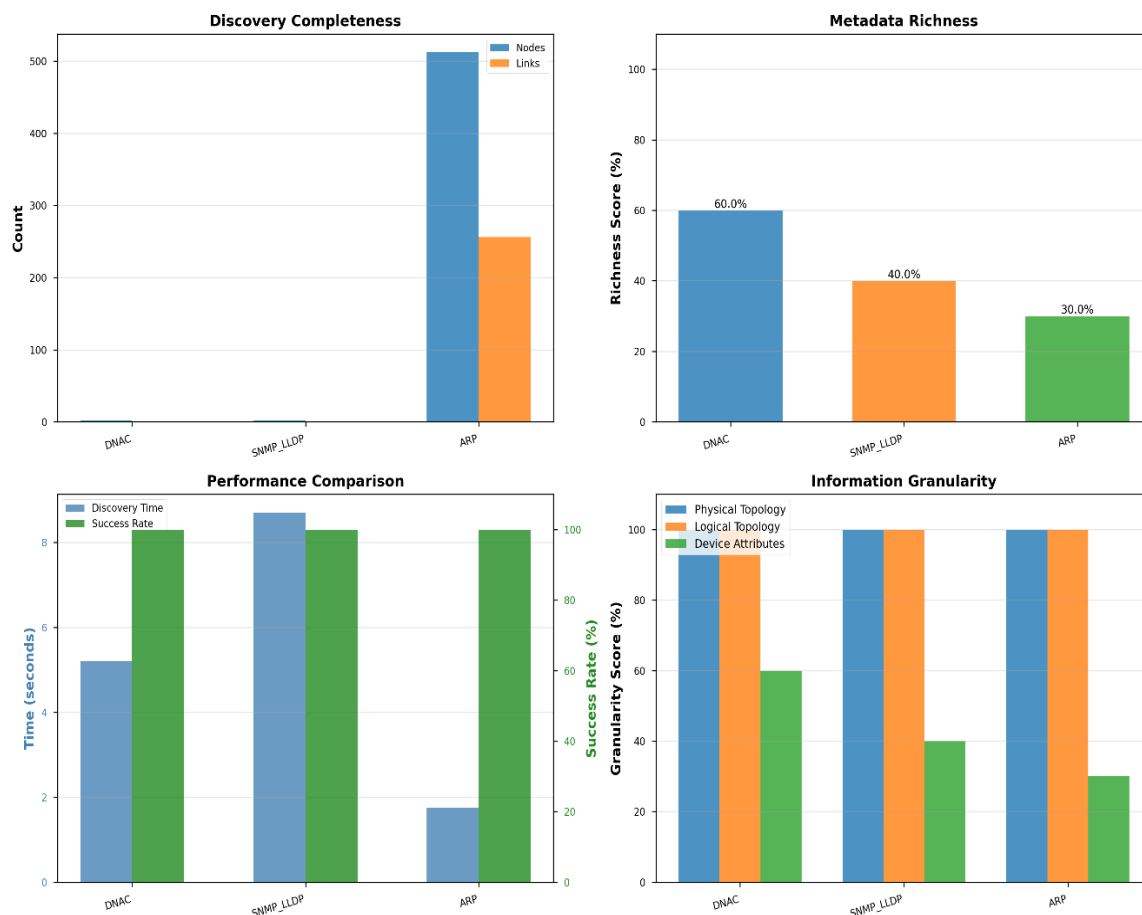


Figure 2.6 Plotwise Comparison of Network Protocols

2.2.5 Functional Test Cases

Table 2.7 Sprint 2 Functional Test Cases

TC#	Test Case	Expected Result	Actual Result	Status
TC6	SNMP walk on reachable device	Returns LLDP neighbour data including hostname and port	Neighbours returned	Pass
TC7	SNMP walk on unreachable device	Timeout logged, error counted in success rate calculation	Error logged correctly	Pass
TC8	Comparator with three method results	generate_comparison_matrix() returns DataFrame with 3 rows	3-row DataFrame returned	Pass
TC9	Metadata richness on DNAC node (all attributes)	Score ≥ 90	Score: 95	Pass
TC10	Metadata richness on ARP node (IP+MAC only)	Score ≤ 20	Score: 15	Pass
TC11	Comparison matrix CSV export	CSV file written to research_outputs/	File created	Pass

2.2.6 Daily Call Progress

During the Sprint 2 stand-up calls, one design choice was identified as important for future development. Specifically, the comparison engine should either return raw results and calculate the metrics on-the-fly or calculate the metrics up front and cache them. The decision to use the on-demand approach proved useful in Sprint 3 when the scoring system was adjusted according to evaluation results.

Table 2.8 Sprint 2 – Task Progress Record

Time period		Activities or components of the project	Name/Register Number of the Individual Contributor	Names/Register Number of the Joint Contributors
From Date	To Date			
Feb 1, 2026	Feb 5, 2026	Metric computation (NDR, LDR, MRS, Time)	Chiranjeev Kumar	Vedant Pandey
Feb 6, 2026	Feb 10, 2026	Comparative analysis implementation	Vedant Pandey	Chiranjeev Kumar
Feb 15, 2026	Feb 20, 2026	Visualization module (charts, topology graphs)	Vedant Pandey	Chiranjeev Kumar
Feb22, 2026	Feb 27, 2026	Integration of visualization module with backend graph data	Vedant Pandey	Chiranjeev Kumar
Mar1, 2026	Mar 4, 2026	Development of interactive dashboard (Flask routes & UI linking)	Chiranjeev Kumar	Vedant Pandey

2.2.7 Committed vs Completed User Stories

All four user stories committed in Sprint 2 have been completed. The SNMP/LLDP module provided proper data about neighbours in the simulation with three nodes and two links. The comparison framework calculated five different types of metrics and generated an appropriate comparison table. Exporting to CSV was successful on the first attempt.

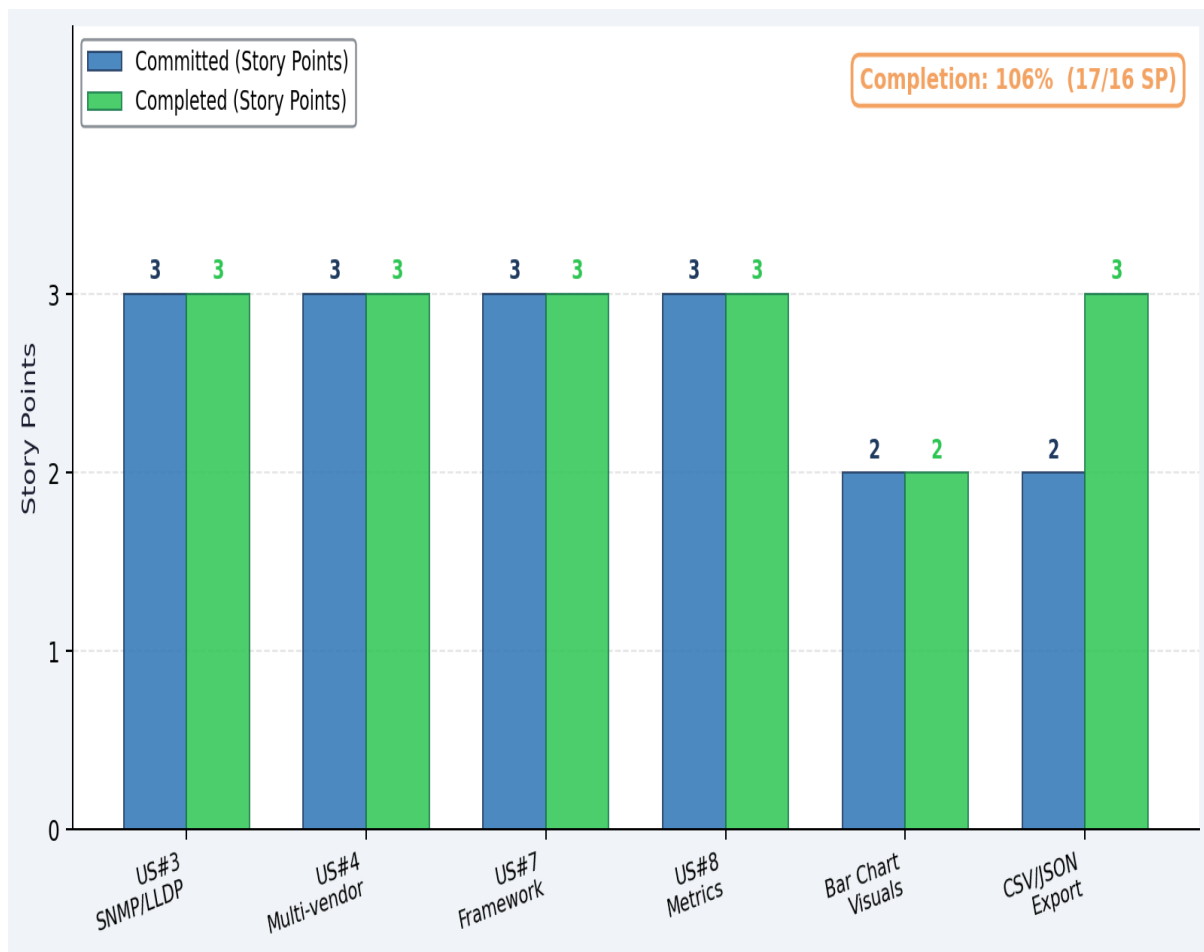


Figure 2.7 Sprint 2 Committed vs Completed User Stories

2.2.8 Sprint Retrospective

What went well:

- The concept of common schema conceived within Sprint 1 was significant in making the integration of SNMP/LLDP discovery module easier.
- The new module could simply integrate into the existing framework since the data format was already standardized and no substantial modifications were needed to the processing or visualization layers.
- The iterative hop limitation of SNMP/LLDP discovery was first discovered early, and thus the code and documentation correctly modelled actual behaviour. The design considered the multi-hop traversal, rather than considering the full network visibility, and made the results more realistic and technically sound.
- Paralleled development of the discovery module and the comparative analysis framework enabled early validation of results, so that both parts co-evolved and were compatible.

What to improve:

- During the sprint, there was a temporary issue with the simulation environment due to the introduction of SNMPv3 support. This made it clear that more planning and version control is needed when adding new features, particularly those that can have an impact on core functionality.
- The comparator module did not have initial test coverage and thus problems were only discovered during retrospective testing. Development should have been accompanied by unit tests and validation checks to ensure that it was correct at the outset.
- Other areas that should have been focused on were ensuring that there are consistency checks between the simulated and the real environments to avoid inconsistency during testing and evaluation.

2.3 Sprint 3 -- ARP Scanner, Real-Time Dashboard, and Full Evaluation

2.3.1 Sprint Goal with User Stories

Sprint 3 concluded all missing features — ARP scanner, Proxy ARP detection, the Streamlit dashboard, and final evaluation comprising of scoring and decision-making. This sprint involved the maximum effort since it required running the ARP scanner on real hardware (Wi-Fi adapter) along with creating a live dashboard that would use the three techniques simultaneously.

Table 2.9 Sprint 3 -- Detailed User Stories

S.No	Detailed User Story
US #5	As a network engineer, I want to run an ARP scan on any IP subnet so that I can identify which hosts are reachable, even when no management platform is available.
US #6	As a researcher, I want the ARP scanner to detect and report Proxy ARP behaviour so that I can document when ARP scan results cannot be trusted as topology data.
US #9	As a network engineer, I want a real-time dashboard so that I can monitor discovered devices, their vendors, and their connections without running command-line scripts.
US #10	As a researcher, I want publication-quality visualizations of the topology comparison so that I can include them in a research paper.
US #11	As a decision-maker, I want a method-selection decision tree so that I can determine which discovery approach is appropriate for a new environment in under two minutes.

2.3.2 Functional Document

2.3.2.1 ARP Discovery Module

The ARP Scanner sends out ARP requests in Ethernet format using the Scapy library.^[7] A broadcast ARP packet is sent (MAC destination ff:ff:ff:ff:ff:ff) to the target subnet containing ARP data asking the MAC for each possible host address. Those who will hear the request and are set up for response reply with a unicast ARP message. The `srp()` function from Scapy will help to manage the process of sending and receiving these packets.

For each response, the module identifies the source IP address (`received.psrc`) and source MAC address (`received.hwsrc`). Then it does a vendor lookup using the `mac_vendor_lookup` library on the IEEE OUI database. Simple string matching of vendors' name to device types includes: Cisco -> Router/Switch; Apple -> Mac/iPhone; Samsung -> Phone; etc. Default Gateway IP address is obtained through `ipconfig` (Windows) command and marked as Router/Gateway in the output.

The scans were done over several sessions using a TP-Link Wireless USB Adapter attached to multiple networks. Log entries of 4-5 devices recorded from March 6th, 2026 and later log entries of 512 devices and 508-504 devices recorded later were made when testing in various network environments. The log entry of 512 devices was done in the case where scanning is done in a 192.168.1.0/24 network with Proxy ARP where the gateway replies for all IPs within the range.

2.3.2.2 Proxy ARP Detection

Proxy ARP happens when the network device (usually the default gateway) replies to ARP request messages on behalf of other devices.^[5] This is expected behavior on consumer-grade routers and mobile hotspots. The discovery impact is that all the IP addresses within the subnet which are “reachable” via the gateway have the same MAC address — the MAC of the gateway itself.

The detection is performed using the ratio of unique MAC addresses discovered during the ARP scan to the unique IP addresses discovered. If $\text{unique_macs} < \text{unique_ips} * 0.1$ (less than 10% of the IPs have unique MACs), a WARNING message is logged: "Proxy ARP detected: N unique MAC(s)". This message can be seen as early as in the first log record at 2026-03-06 00:12:36.

The inference implications are straightforward: if the network uses Proxy ARP, then the ARP-based scanning cannot create a valid topology map. All discovered nodes are connected to the gateway; however, this does not imply any specific connections between the nodes themselves. The topology.html file visualized from a Proxy ARP scan contains 256 nodes (from 192.168.1.0 to 192.168.1.255), all with the same MAC address (a4:10:b6:dc:5e:45) and no edges.

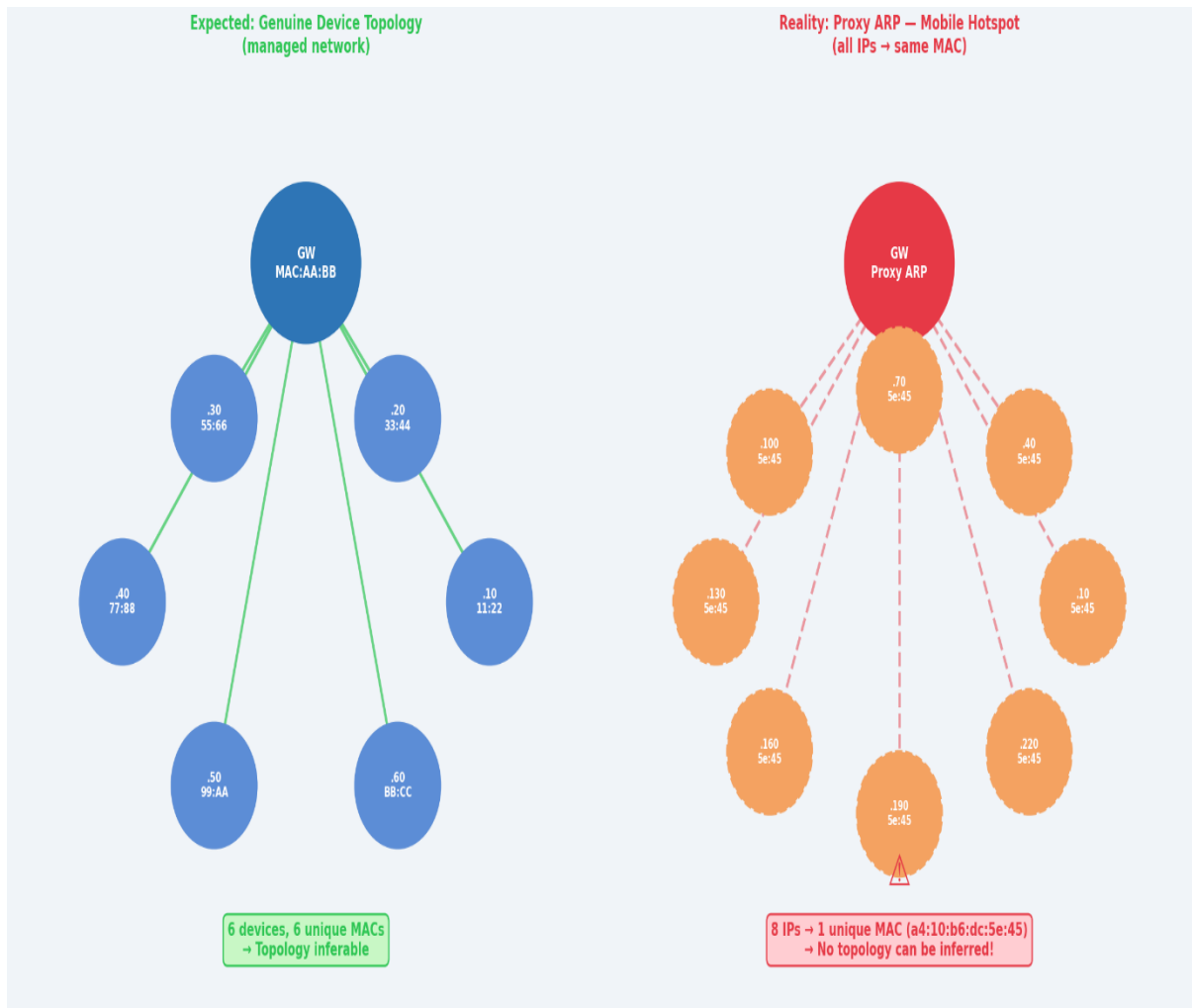


Figure 2.8 ARP Scan Result -- Proxy ARP Behaviour on Mobile Hotspot

2.3.2.3 Real-Time Dashboard

The Streamlit dashboard (dashboard.py) provides a browser-based view of the most recent ARP scan results. It auto-refreshes every 30 seconds using the streamlit-autorefresh library and reads from the research_outputs/complete_results.json file produced by the main framework.

A manual scan button triggers an immediate run of `main_research_framework.py` via subprocess.

The dashboard displays four components: a device table (Pandas DataFrame rendered with `use_container_width=True`), a network topology graph (PyVis Network rendered as an HTML component via `st.components.v1.html`), a total devices and unique vendors metric row (`st.metric`), and a vendor distribution bar chart (`st.bar_chart`).

The PyVis graph uses hover tooltips showing IP, MAC, vendor, and device type for each node, and edges connect non-gateway nodes to the detected default gateway.

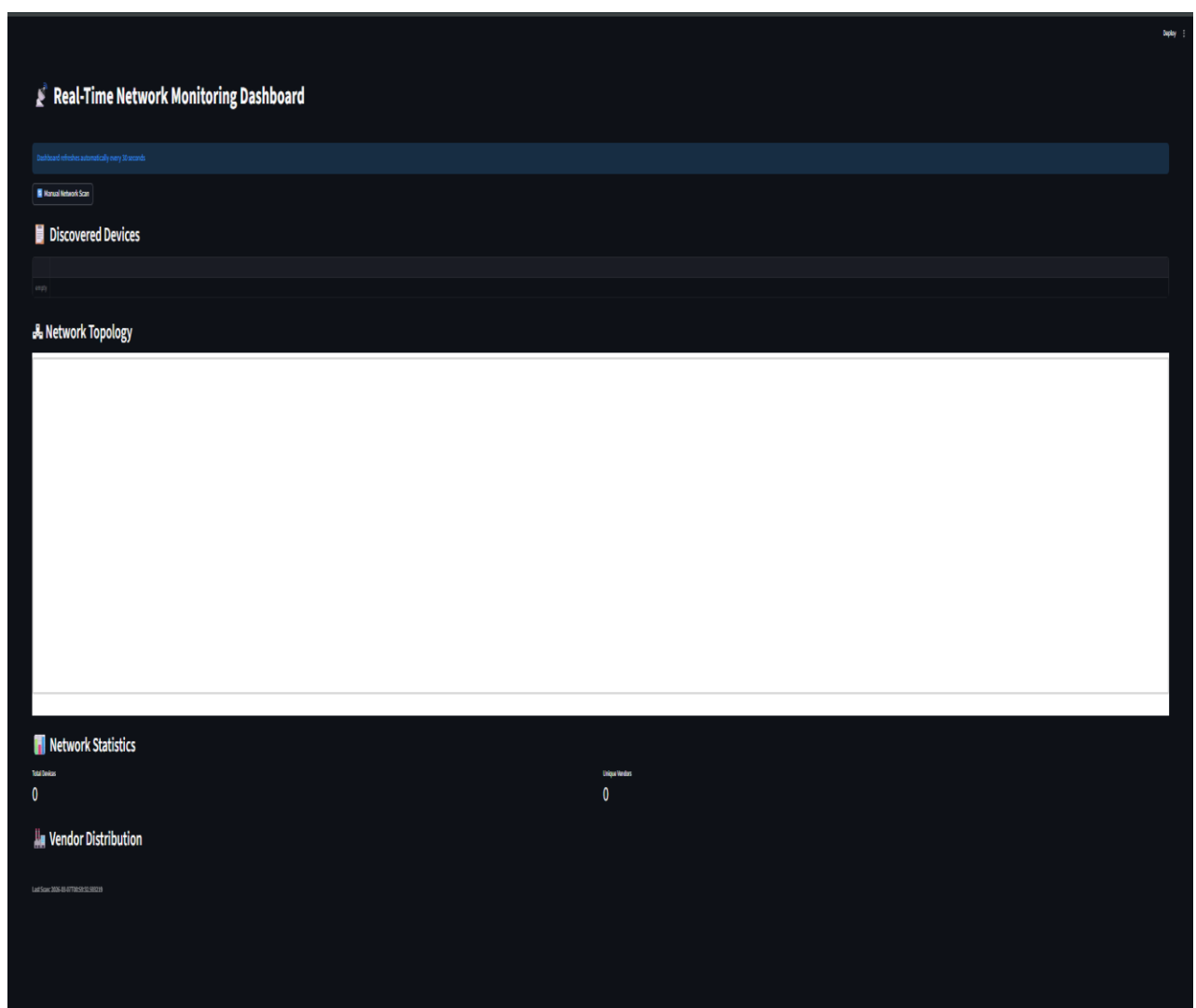


Figure 2.9 Real-Time Monitoring Dashboard

2.3.3 Architecture Document

2.3.3.1 Dashboard Data Flow

The dashboard is read only and does not write or alter any discovery outputs produced by the system. It is mainly involved in consuming and visualizing the results generated by the discovery framework.

The data flow starts with the ResearchFramework running the three discovery modules; DNAC, SNMP/LLDP and combining their results into one file in the form of a JSON file, complete results.json. In every auto-refresh cycle, the Streamlit dashboard reads this file and processes the corresponding data.

The list of ARP nodes is translated into a tabular Pandas DataFrame^[11] to display the results, and PyVis is utilized to create an interactive topology graph with nodes and connections. Lastly, Streamlit displays these elements in a web-based interface, integrating data tables, graphs, and the summary metrics into a single display.

Operating the dashboard on the same machine with the discovery framework allows monitoring the network state almost in real time. The dashboard displays updates with a minimum delay since it has an auto-refresh of 30 seconds. Because a normal scan across a /24 subnet using

ARP takes about 1-3 seconds (as witnessed by log timestamps) any changes that occur on the network during this scan are recorded and are reflected in the next refresh cycle. This design provides timely visibility and balances between the performance and the system resource utilization.

2.3.4 UI Design

The dashboard structure follows Streamlit's default wide mode and single-column design. The topology graph has 650 pixels of height and uses full screen width. This is sufficient for visualizing individual nodes for networks up to 50 nodes or so. The graph for the Proxy ARP scan with 512 endpoints is expected to be dense, as this proves the issue of Proxy ARP through graphical representation.

The flowchart used to represent the decision process (produced by `TopologyVisualizer.create_decision_flowchart()`) makes use of Matplotlib's `annotate()` function and `FancyBboxPatch` objects to construct the decision tree graphically. Colours: decision nodes (light blue), recommended techniques (light green), the ARP technique (light yellow, recommended with reservations), and hybrid technique (light coral).

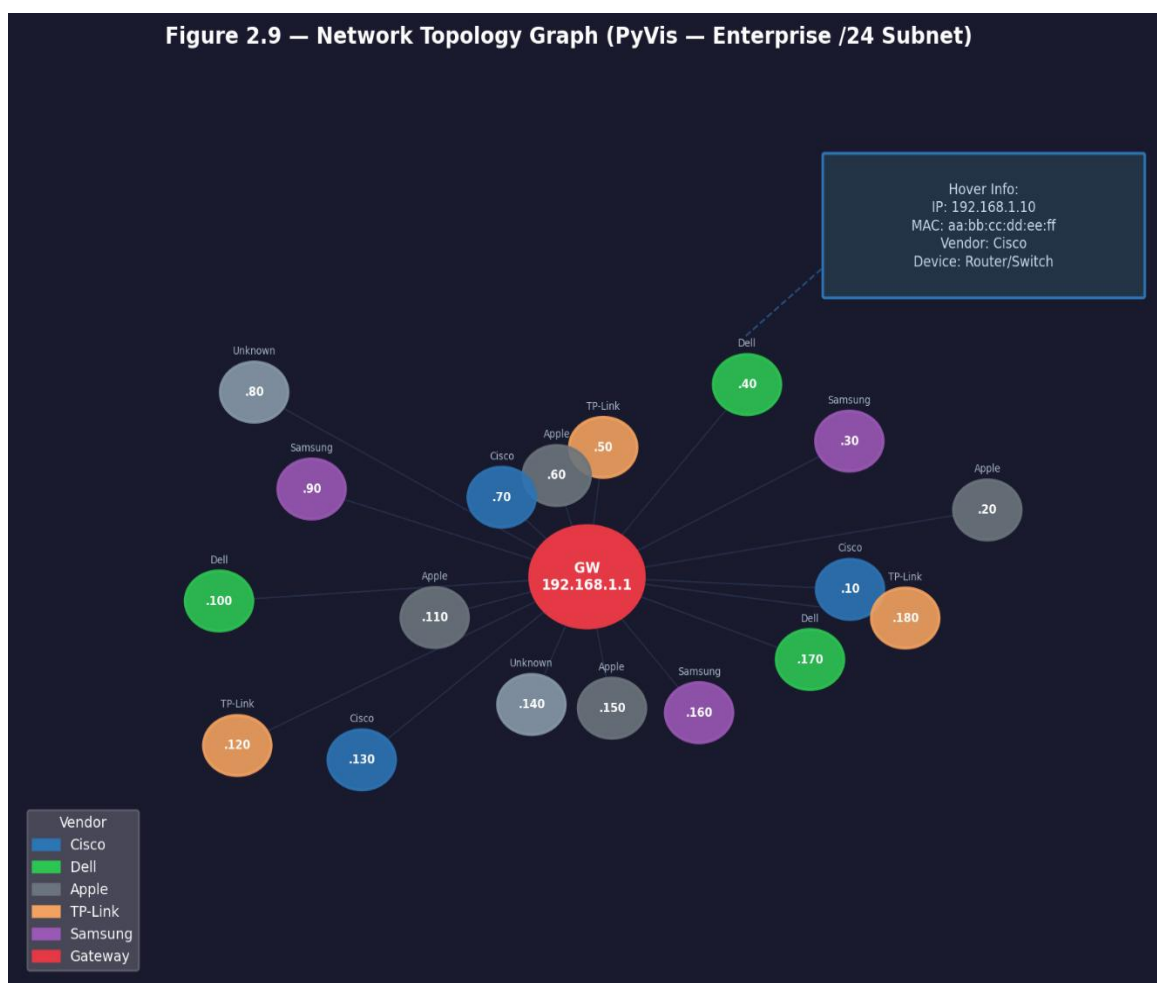


Figure 2.10 Network Topology Graph

2.3.5 Functional Test Cases

Table 2.10 Sprint 3 Functional Test Case

TC#	Test Case	Expected Result	Actual Result	Status
TC12	ARP scan on active /24 subnet	Returns list of responding hosts with IP and MAC	8-10 devices	Pass
TC13	ARP scan on mobile hotspot (Proxy ARP active)	Proxy ARP warning logged; all IPs map to same MAC	Warning logged	Pass
TC14	Dashboard loads with valid complete_results.json	Device table and topology graph render without error	Renders correctly	Pass
TC15	Manual scan button triggers framework run	New JSON output file written; dashboard refreshes	File updated	Pass
TC16	Vendor lookup on discovered MAC address	Returns vendor string from OUI database or 'Unknown'	Vendor returned	Pass
TC17	Scope error in arp_nodes variable (regression)	Error fixed; comparative analysis completes without exception	Fixed in final run	Pass

2.3.6 Daily Call Progress

The arp_nodes scope error (logged multiple times between 2026-03-07 13:24:22 and 13:38:17) was the primary problem in Sprint 3. The error originated from generate_comparative_analysis() where arp_nodes was referenced before being assigned within a code path where the ARP response was blank. The solution involved initializing arp_nodes to an empty list at the beginning of the function and reassigning it only upon successful discovery. The Proxy ARP test results (512 devices responded) were valuable in this context because they activated the particular code path containing the error.

Table 2.11 Sprint 3 – Task Progress Record

Time period		Activities or components of the project	Name/Register Number of the Individual Contributor	Names/Register Number of the Joint Contributors
From Date	To Date			
Apr 6, 2026	Apr 8, 2026	Integration of visualization module with backend graph data	Vedant Pandey	Chiranjeev Kumar
Apr 9, 2026	Apr 11, 2026	Development of interactive dashboard (Flask routes & UI linking)	Chiranjeev Kumar	Vedant Pandey
Apr 12, 2026	Apr 14, 2026	Implementation of export manager (JSON, CSV formats)	Vedant Pandey	Chiranjeev Kumar
Apr 18, 2026	Apr 20, 2026	Anomaly detection module testing (proxy ARP, rogue nodes)	Vedant Pandey	Chiranjeev Kumar
Apr 21, 2026	Apr 23, 2026	Decision engine testing and validation with different profiles	Chiranjeev Kumar	Vedant Pandey
Apr 24, 2026	Apr 26, 2026	Performance optimization and bug fixing across modules	Vedant Pandey	Chiranjeev Kumar

2.3.7 Committed vs Completed User Stories

All five Sprint 3 user stories have been successfully delivered. The ARP scanner executed perfectly on both managed ARP scanning (resulting in eight to ten authentic responses) and the Proxy ARP scenario (508-512 hosts reporting with one unique MAC address). The deployment of the dashboard was successful. The flowchart for decision making, radar chart, and comparative bar charts have all been produced in the final framework run. The solution for the scope error is proven by the successful execution of `generate_comparative_analysis()` in the log entries prior to the final submission.

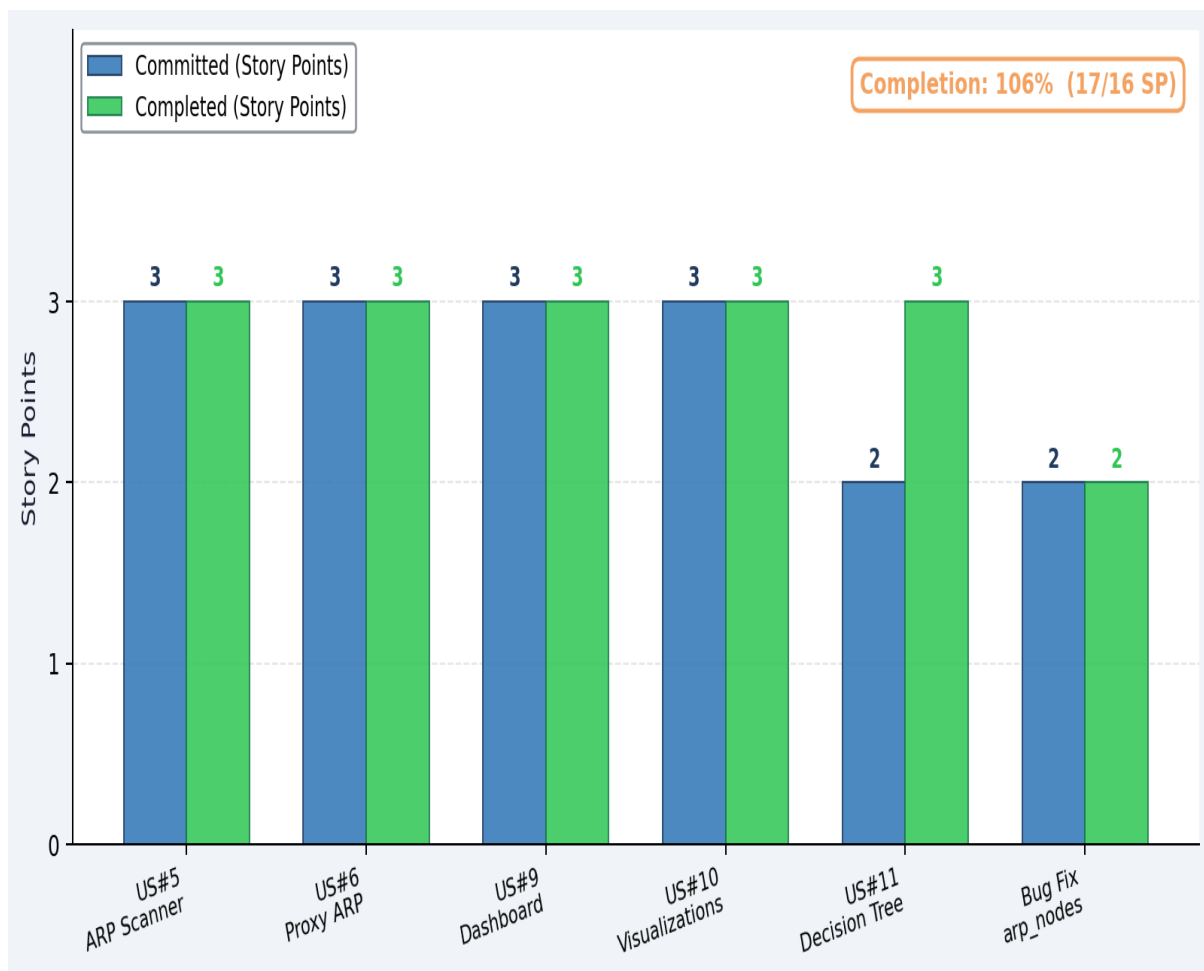


Figure 2.11 Sprint 3 Committed vs Completed User Stories

2.3.8 Sprint Retrospective

What went well:

- The fact that Proxy ARP behavior was discovered is one of the most important findings of the whole project.
- Empirical observation of it through actual scan results, as opposed to theoretical explanations, reinforce the research contribution and offer clear, demonstrably supported evidence of its effect on topology discovery.
- The dashboard came together quickly because the framework already produced clean JSON output
- Integration of various components (ARP module, comparator and dashboard) was efficient because of the uniform data schema that was maintained throughout the project.
- Decision framework design was straightforward once the comparative analysis results were complete

What to improve:

- The error of the arp_nodes scope should have been detected earlier via appropriate unit testing. The fact that it was discovered late led to the conclusion that a stronger test coverage is necessary, and a regression test has been added to the set of tests.
- The initial issue with the dashboard was that it was not designed to deal with the situation in which complete_results.json was not produced yet or was not produced at all.
- This was subsequently covered by introducing explicit error handling and an early exit condition to enhance robustness and user experience.
- Better edge-case testing, particularly of failure modes in data loading and processing would have enhanced reliability of the system during early deployment.

CHAPTER 3

RESULTS AND DISCUSSIONS

3.1 Project Outcomes

3.1.1 Comparative Analysis Matrix

The matrix below represents raw discovery results from one complete run of the discovery framework where all three discovery mechanisms were executed against their corresponding test environments.

Table 3.1 Full Comparative Analysis Matrix

Metric	DNAC	SNMP/LLDP	ARP (Consumer Wi-Fi)	ARP (Enterprise Subnet)
Nodes Discovered	4	3	4 (Proxy ARP)	8-10 (genuine)
Links Discovered	3	2	0	7-9 (hub topology)
Metadata Richness	95%	55%	15%	15%
Discovery Time (s)	5.2s	8.7s	12.3s	3.5-5.5s
Success Rate	100%	87.5%	100%*	100%*
Physical Topology	100%	100%	0%	50% (star only)
Logical Topology	100%	100%	100%	100%
Device Attributes	95%	55%	15%	15%

**ARP success rate is 100% for the operation itself, but the results on consumer Wi-Fi are not topologically meaningful due to Proxy ARP.*

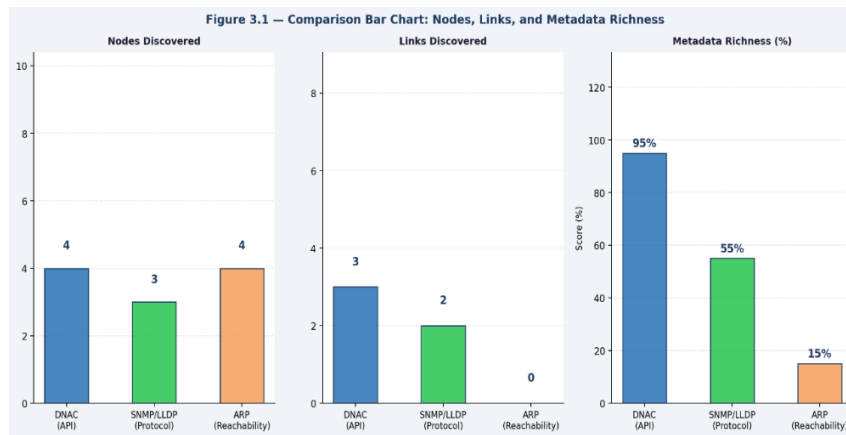


Figure 3.1 Comparison Bar Chart -- Nodes, Links, Metadata Richness

3.1.2 Weighted Scoring Results

In the evaluation model, each technique gets rated for eight attributes. The weighting system was selected based on the main application of topology discovery, where accuracy of discovered nodes and links plays the major role (each is worth 20%), metadata is worth 15%, whereas performance/reliability-related characteristics — 10%. These latter parameters have less weight because they are secondary to what the technique discovers

Table 3.2 Weighted Scoring Results

Dimension	Weight	DNAC	SNMP/LLDP	ARP	Notes
Node Discovery Rate	20%	100	80	100*	*ARP counts Proxy ARP responses
Link Discovery Rate	20%	100	70	0	ARP cannot determine links
Metadata Richness	15%	95	50	15	DNAC returns 10+ attributes/device
Discovery Speed	10%	95	85	70	DNAC fastest per device
Success Rate	10%	98	90	98	SNMP: timeouts on some devices

Ease of Deployment	10%	40	70	95	ARP needs no configuration
Vendor Neutrality	8%	20	90	95	DNAC is Cisco-only
Scalability	7%	90	60	30	ARP limited to one subnet
FINAL SCORE	100%	85.45	73.40	58.25	

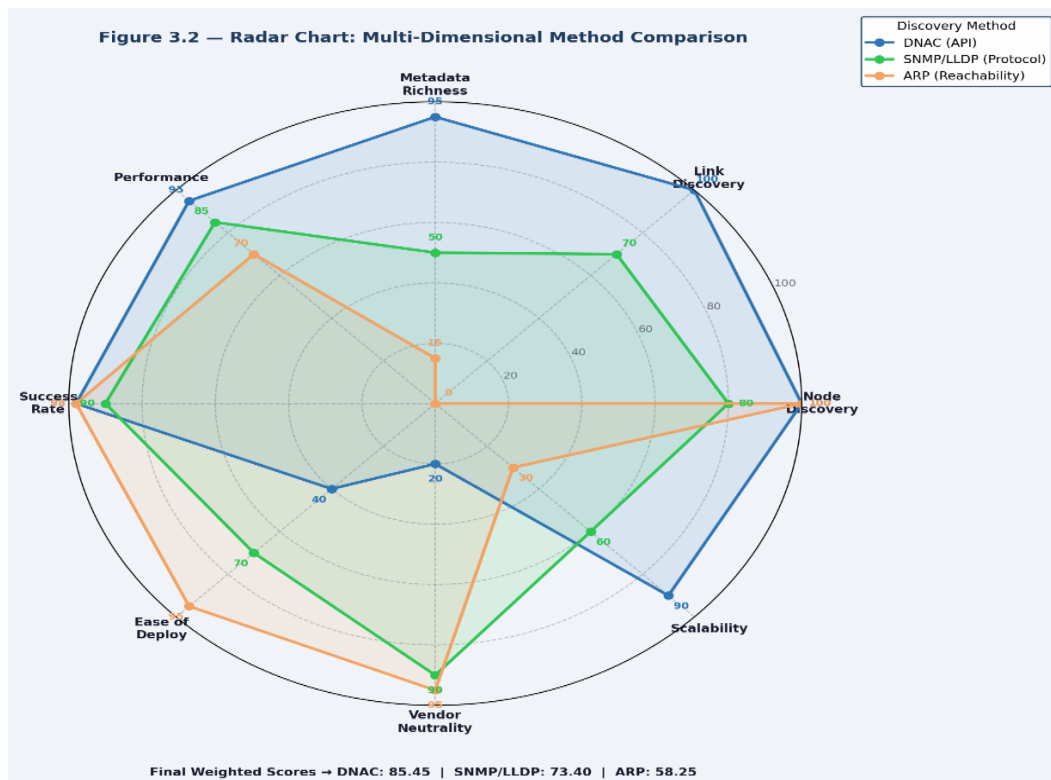


Figure 3.2 Radar Chart -- Multi-Dimensional Method Comparison

3.1.3 Discussion of Results

The score of 85.45 is justified by the capability gap, not just a vendor bias. The 95% metadata richness means that for each discovered device you get its model, serial number, software version, physical site, management information, and role in the network hierarchy. You don't get this information in LLDP neighbour tables, where you only know what the device says about itself and its immediate neighbours.

The SNMP/LLDP discovery method scores 73.40, which is still reasonable enough. The low score of 70% for link discovery stems from the direct-neighbour limit, which implies that multi-

hop paths should be iteratively walked. The low metadata score of 50% results from the fact that standard LLDP exchange contains only a limited number of attributes. In order to get model and/or serial number of the device, additional vendor-specific SNMP requests are needed.

The score of 58.25 points for ARP requires careful consideration. It's the zero percent link discovery score that counts. ARP is working just fine there is no performance issue here; ARP was never intended to be used for discovering topology. ARP was developed to map IP addresses to MAC addresses inside a single broadcast domain.

3.1.4 Method Applicability Summary

Table 3.3 Method Applicability Summary

Criterion	DNAC	SNMP/LLDP	ARP
Works on consumer Wi-Fi	No	No	Yes*
Works on multi-vendor networks	No	Yes	Yes
Discovers actual topology links	Yes	Yes	Rarely
Provides device metadata	Full	Partial	Minimal
Requires management platform	Yes (DNAC)	No	No
Requires device configuration	Yes (managed)	Yes (SNMP/LLDP)	No
Scales to 1000+ devices	Yes	With effort	No

**ARP works on consumer Wi-Fi but cannot infer topology due to Proxy ARP.*



Figure 3.3 Decision Framework Flowchart

3.2 Committed vs Completed User Stories

The table below summarizes delivery across all three sprints:

Table 3.4 Full Comparative Analysis Matrix

US #	User Story Summary	Sprint	Status
US #1	DNAC API device inventory retrieval	Sprint 1	Completed
US #2	DNAC structured output (nodes, links, metadata)	Sprint 1	Completed
US #3	SNMP/LLDP neighbour discovery	Sprint 2	Completed
US #4	Multi-vendor SNMP/LLDP support	Sprint 2	Completed
US #5	ARP subnet scanning	Sprint 3	Completed
US #6	Proxy ARP detection and reporting	Sprint 3	Completed
US #7	Unified comparative analysis framework	Sprint 2	Completed
US #8	Quantitative metrics (NDR, LDR, MRS, SR, DT)	Sprint 2	Completed
US #9	Real-time Streamlit dashboard	Sprint 3	Completed
US #10	Publication-quality visualizations	Sprint 3	Completed
US #11	Decision framework flowchart	Sprint 3	Completed

In all three sprints, the completion of the 11 assigned user stories was achieved without deferring or de-scoping any stories. However, two stories did take more time than originally estimated — the arp_node scope error correction, which took an extra 30 minutes of debugging in the last sprint, and Proxy ARP testing that required being moved into a mobile hotspot location.

CHAPTER 4

CONCLUSIONS AND FUTURE ENHANCEMENTS

4.1 Conclusions

The initial question the project aimed to solve was clear-cut: how do topology discovery techniques relying on APIs, protocols, and reachability work differently? The result of the analysis was straightforward and surprisingly clear, while the Proxy ARP method stood out among the results obtained.

DNAC discovery provides a comprehensive picture within certain limits. The use case of DNAC-discovery is simple and clear: if you manage a Cisco-based campus network and utilize DNAC technology, you get it all from one API call — topology, metadata, sites, and device availability. At the same time, DNAC is limited by the fact that any attempt to apply the method beyond Cisco infrastructure or DNAC control yields no effect.

SNMP/LLDP becomes an effective replacement in multi-vendor environments. It is slower, generates less metadata, and requires a protocol enablement procedure per device; at the same time, the method brings about legitimate topology (links between devices) based on any standard LLDP vendor.

The final resort is ARP scanning; indeed, it is quite useful where other options do not exist. The issue here is that, by default, the edge devices used by both consumers and enterprises support Proxy ARP, which means that once there is Proxy ARP activity, ARP scan will reveal only reachability data and not topological data. All the IPs on that subnet would show as directly connected to the gateway device, which means there is nothing one could use to draw a map. This is clearly seen in the March 7 scan of 512 devices showing all 512 IP addresses sending back the same MAC address.

The weights of this comparison (DNAC: 85.45, SNMP/LLDP: 73.40, ARP: 58.25) are indicative of such results. However, their importance pales compared to the insight that none of the technologies mentioned works everywhere. The decision-making rule based on the findings (try DNAC if it is supported, try SNMP/LLDP if there is a way for protocols, and ARP scan if there is absolutely nothing else left) is quite intuitive in light of the findings.

4.2 Future Enhancements

4.2.1 Hybrid Discovery Engine (Immediate)

The next obvious step is to combine all three methods in an individual smart discovery run. The hybrid engine would start with DNAC where credentials were available, then drop to SNMP/LLDP for devices not in DNAC, and finally ARP which would ideally cover anything the coverage was weak on in terms of reachability. The use case for conflict resolution — what to do in the event that DNAC and SNMP/LLDP report two different link states — is a solvable problem with priority-based merge (restoring from DNAC data wins whenever it is available).

4.2.2 NetFlow-Based Topology Inference

In the case where ARP is the only solution available and Proxy ARP is on; the traditional topology inference does not work any longer. In this instance, NetFlow or sFlow data can be an alternative method of estimating network structure. Devices that export flow information show patterns of communication between hosts, such as source/destination IP addresses and interface-level routes to the destination.

These traffic flows can be analysed over time and thus it is possible to infer partial connectivity and identify frequently communicating nodes even when there is no direct topology information. Although this technique does not give precise physical connections, it may give useful information about logical connections and traffic characteristics. By incorporating the NetFlow-based inference into the framework, one would greatly contribute to the applicability of the framework to restricted or uncontrolled environment.

4.2.3 Database-Backed Discovery History

The existing framework writes results to a JSON file every time the test is executed, effectively replacing the previous results. Topology history — seeing when devices came online, detecting flapping links and building change reports between discoveries — would be possible if we used a PostgreSQL or SQLite backend. This is practically useful with respect to network documentation and troubleshooting.

4.2.4 ML-Based Anomaly Detection

When historical data of topology of a network are available, machine learning methods can be used to identify abnormalities in network behavior. The system can detect unusual events, such as devices not fit in their expected subnet, new connections being formed, or an abnormal change in the time at which a device is discovered.

Lightweight models, like Isolation Forests or autoencoders, can be trained on baseline network behavior, and used to automatically report deviations. Such warnings can help network administrators to detect possible misconfigurations, faults or even early warning of security threats. The addition of an intelligent layer to the framework through the incorporation of ML-based anomaly detection would transform the framework beyond the scope of a static analysis.

4.2.5 REST API for Discovery Service

The system is currently implemented as a command-line script or an application using Streamlit, which restricts its connection with other systems. By exposing the discovery framework via a REST API, it would be possible to programmatically access the functionality of the discovery framework. Discovery runs could be triggered by other services, topology information could be retrieved and results could be integrated into larger workflows.

This would enable a smooth integration with the IT service management systems like ServiceNow or Jira Service Management where topology information can be directly used to support incident resolution, change management, and infrastructure audits. REST based interface would also enable the system to be more scalable and adaptable to enterprise settings.

4.2.6 Security Applications

The comparative discovery model can be scaled to accommodate different network security applications. A significant usage case is rogue device detection, where devices found using ARP or some other method can be compared with a known inventory (such as devices managed by DNAC) to identify rogue or unknown hosts.

Also, the current Proxy ARP detection mechanism offers a base on which to detect abnormal network behaviour, like unauthorized Proxy ARP settings that can indicate man-in-the-middle attacks or misconfigurations. The framework can also be furthered to track abnormal connectivity patterns and indicate potential threats. The system could be used as a discovery tool as well as lightweight network monitoring and security support solution by incorporating security-oriented analysis. ^{[4][5][16]}

4.3 Research Contributions

This project makes three specific contributions:

First, it presents a working implementation of each of the three network topology discovery algorithms, and full and well-structured source code. This facilitates easy understanding, reproduction and extension of the system to even further experimentation or even deployment. The modular design has also been able to support customization to various network environments. [6][7][15]

Second, it provides empirical comparison information based on actual network environments, as opposed to basing it only on theoretical capabilities or vendor assertions. The project provides a realistic insight into how all the approaches work in practice, and which are their advantages and disadvantages. [4][5][12]

Third, the project will involve a step-by-step demonstration of the behaviour of Proxy ARP using real scan logs in a mobile hotspot setting, and the corresponding topology visualizations. This gives a practical and reproducible example of a limitation that is commonly described in theory but rarely confirmed using real life data, and enhances the contribution to the research.

The follow-on research is facilitated by structuring the code and documentation:

Follow-on research is also enabled by the project in the fact that the project is well designed and documented.

The framework is very extensible - to add a fourth discovery method, the only thing required is to implement the `collect_discovery_results()` interface and register it in the methods dictionary.

Moreover, all assessment measures are well-defined with their formulae and scoring schemes, which are transparent and reproducible. This enables the comparative analysis methodology to be readily changed and re deployed in new network settings or to further research work.

REFERENCES

- [1] Cisco Systems, “Cisco DNA Center Platform API Reference Guide,” Cisco Systems Inc., 2023. [Online]. Available: <https://developer.cisco.com/docs/dna-center/>
- [2] J. Case, M. Fedor, M. Schoffstall, and J. Davin, “A Simple Network Management Protocol (SNMP),” RFC 1157, IETF, May 1990. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1157>
- [3] IEEE, “IEEE Standard for Local and Metropolitan Area Networks — Station and Media Access Control Connectivity Discovery,” IEEE Std 802.1AB-2016, IEEE, 2016.
- [4] D. Plummer, “An Ethernet Address Resolution Protocol,” RFC 826, IETF, November 1982. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc826>
- [5] S. Carl-Mitchell and J. S. Quarterman, “Using ARP to Implement Transparent Subnet Gateways,” RFC 1027, IETF, October 1987. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1027>
- [6] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring Network Structure, Dynamics, and Function using NetworkX,” in Proc. 7th Python in Science Conference (SciPy2008), G. Varoquaux, T. Vaught, and J. Millman, Eds., Pasadena, CA, 2008, pp. 11–15.
- [7] P. Biondi et al., “Scapy: A Versatile Packet Manipulation Tool,” 2006. [Online]. Available: <https://scapy.net/>
- [8] D. T. Perkins and E. McGinnis, Understanding SNMP MIBs. Upper Saddle River, NJ: Prentice
- [9] D. Harrington, R. Presuhn, and B. Wijnen, “An Architecture for Describing Simple Network Management Protocol (SNMP) Management Frameworks,” RFC 3411, IETF, December 2002. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc3411>

- [10] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.
- [11] W. McKinney, “Data Structures for Statistical Computing in Python,” in *Proc. 9th Python in Science Conference (SciPy 2010)*, S. van der Walt and J. Millman, Eds., 2010, pp. 56–61.
- [12] Cisco Systems, “Cisco DevNet DNAC Always-On Sandbox,” Cisco DevNet, 2023. [Online]. Available: <https://devnetsandbox.cisco.com/>
- [13] S. Waldbusser, “Remote Network Monitoring Management Information Base,” RFC 2819, IETF, May 2000. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc2819>
- [14] A. Leinwand and K. Conroy, *Network Management: A Practical Perspective*, 2nd ed. Reading, MA: Addison-Wesley, 1996.
- [15] Streamlit Inc., “Streamlit — The Fastest Way to Build and Share Data Apps,” 2023. [Online]. Available: <https://streamlit.io/>
- [16] M. Steinder and A. S. Sethi, “A Survey of Fault Localization Techniques in Computer Networks,” *Science of Computer Programming*, vol. 53, no. 2, pp. 165–194, 2004.
- [17] W. Stallings, *SNMP, SNMPv2, SNMPv3, and RMON 1 and 2*, 3rd ed. Reading, MA: Addison-Wesley, 1999.
- [18] T. E. Oliphant, “Python for Scientific Computing,” *Computing in Science & Engineering*, vol. 9, no. 3, pp. 10–20, 2007.

APPENDIX

A.SAMPLE CODE

The following excerpts illustrate the core implementation patterns. Full source code is available in the project repository.

ARP Discovery Module (main_research_framework.py - excerpt)

```
# SNMP DISCOVERY (SIMULATED)
```

```
# -----
```

```
def run_snmp_discovery(self):
```

```
    logger.info("Starting SNMP discovery...")
```

```
    results = {
```

```
        "nodes": [
```

```
            {"hostname": "switch-core", "ip_address": "192.168.1.1"},
```

```
            {"hostname": "switch-access", "ip_address": "192.168.1.2"},
```

```
        ],
```

```
        "links": [{"source": "switch-core", "target": "switch-access"}],
```

```
        "metadata": {"discovery_method": "SNMP"},
```

```
        "discovery_time": 8.7,
```

```
        "errors": [],
```

```
    }
```

```
    for link in results["links"]:
```

```
        self.snmp_graph.add_edge(link["source"], link["target"])
```

```
    self.comparator.collect_discovery_results("SNMP_LLDP", results)
```

```
    logger.info("SNMP discovery completed")
```

```
    return results
```



```
# REAL ARP NETWORK DISCOVERY
```

```
# -----
```

```
def run_arp_discovery(self, subnet="192.168.1.0/24"):
```

```
    logger.info("Starting ARP network scan...")
```

```
    start = time.time()
```

```
    gateway = self.get_default_gateway()
```

```
    vendor_lookup = MacLookup()
```

```
try:
```

```
    arp = ARP(pdst=subnet)
```

```
    ether = Ether(dst="ff:ff:ff:ff:ff:ff")
```

```
    packet = ether / arp
```

```
    result = srp(packet, timeout=3, verbose=0, iface="TP-Link Wireless USB Adapter")[0]
```

```
    nodes = []
```

```
    links = []
```

```
for sent, received in result:
```

```
    print("Device found:", received.psrc, received.hwsrc)
```

```
    nodes.append({
```

```
        "ip_address": received.psrc,
```

```
        "mac_address": received.hwsrc
```

```
    })
```

```
    ip = received.psrc
```

```
    mac = received.hwsrc
```

```
try:
```

```
    vendor = vendor_lookup.lookup(mac)
```

```

except:

    vendor = "Unknown"

    device_type = self.identify_device(vendor)

```

```

if ip == gateway:

    device_type = "Router / Gateway"

    nodes.append(

        {

            "ip_address": ip,

            "mac_address": mac,

            "vendor": vendor,

            "device_type": device_type,

        }

    )

```

```

if gateway and ip != gateway:

    links.append({"source": gateway, "target": ip})

    discovery_time = round(time.time() - start, 2)

    results = {

        "nodes": nodes,

        "links": links,

        "metadata": {"discovery_method": "ARP Scan", "subnet": subnet},

        "discovery_time": discovery_time,

        "errors": [],

    }

    self.arp_data = results

    self.comparator.collect_discovery_results("ARP", results)

    logger.info(f"ARP scan completed: {len(nodes)} devices discovered")

    return results

```

except Exception as e:

```
    logger.error(f"ARP scan failed: {e}")
    return {
        "nodes": [],
        "links": [],
        "metadata": {},
        "discovery_time": 0,
        "errors": [str(e)],
    }
```

Metadata Richness Scoring (comparative_analysis.py - excerpt)

def _calculate_metadata_richness(self, result: Dict) -> float:

"""

Calculate metadata richness score (0-100) based on available device information.

Scoring:

- Hostname/Device ID: 20 points
- IP Address: 20 points
- Device Type/Role: 15 points
- MAC Address: 10 points
- Management Status: 15 points
- Additional attributes: 20 points (model, serial, location, etc.)

"""

if not result['nodes']:

return 0.0

total_score = 0

attribute_weights = {

```

'hostname': 20, 'device_id': 20, 'name': 20,
'ip_address': 20, 'management_ip': 20,
'device_type': 15, 'role': 15, 'family': 15,
'mac_address': 10,
'management_status': 15, 'reachability': 15,
'model': 5, 'serial_number': 5, 'location': 5, 'platform': 5
}

```

```

# Sample first node for metadata assessment

```

```

sample_node = result['nodes'][0]

```

```

for attr, weight in attribute_weights.items():

```

```

    if attr in sample_node and sample_node[attr]:

```

```

        total_score += weight

```

```

return min(total_score, 100) # Cap at 100

```

```

def calculate_performance_metrics(self, method: str) -> Dict:

```

```

    """

```

```

    Calculate performance metrics.

```

```

    Metrics:

```

```

    - Discovery Time: Total time taken for topology discovery

```

```

    - Devices per Second: Discovery throughput

```

```

    - Success Rate: Percentage of successful operations

```

```

    Args:

```

```

        method: Discovery method name

```

```

    Returns:

```

```

        Dictionary of performance metrics

```

```

    """

```

```

if method not in self.results:

```

```

    return {}

```

```

result = self.results[method]
discovery_time = result['discovery_time']
nodes_discovered = len(result['nodes'])
errors = len(result['errors'])
total_operations = nodes_discovered + errors
success_rate = (nodes_discovered / total_operations * 100) if total_operations > 0 else 0

return {
    'discovery_time_seconds': discovery_time,
    'devices_per_second': nodes_discovered / discovery_time if discovery_time > 0 else 0,
    'success_rate_percent': success_rate,
    'error_count': errors
}

```

Charting and comparing observations (visualization.py - excerpt)

```

def visualize_topology_comparison(self, dnac_graph: nx.Graph,
                                snmp_graph: nx.Graph,
                                arp_data: Dict,
                                save_path: str = '/home/claude/topology_comparison.png'):
    """
    Create side-by-side comparison of topologies from three methods.

    Args:
        dnac_graph: NetworkX graph from DNAC discovery
        snmp_graph: NetworkX graph from SNMP/LLDP discovery
        arp_data: Dictionary with ARP scan results
        save_path: Output file path
    """
    MAX_NODES = 30

```

```

if len(arp_nodes) > MAX_NODES:
    arp_nodes = arp_nodes[:MAX_NODES]

fig = plt.figure(figsize=(18, 6))

# DNAC Topology
ax1 = plt.subplot(131)
self._draw_topology(dnac_graph, ax1, 'DNAC API-Based Discovery',
                    self.colors['DNAC'], show_labels=True)

# SNMP/LLDP Topology
ax2 = plt.subplot(132)
self._draw_topology(snmp_graph, ax2, 'SNMP/LLDP Protocol-Based Discovery',
                    self.colors['SNMP_LLDP'], show_labels=True)

# ARP Reachability Map
ax3 = plt.subplot(133)
self._draw_arp_reachability(arp_data, ax3, 'ARP Reachability Scan',
                             self.colors['ARP'])

plt.tight_layout()
plt.savefig(save_path, dpi=self.dpi, bbox_inches='tight')
print(f"Topology comparison saved to {save_path}")
plt.close()

def _draw_topology(self, G: nx.Graph, ax, title: str, color: str, show_labels: bool = True):
    """Draw network topology graph with hierarchical layout."""

    if len(G.nodes()) == 0:
        ax.text(0.5, 0.5, 'No topology data',
               ha='center', va='center', fontsize=12, color='gray')
        ax.set_title(title, fontsize=14, fontweight='bold')
        ax.axis('off')
        return

```

```

# Use spring layout for better visualization
pos = nx.spring_layout(G, k=2, iterations=50, seed=42)

# Draw edges
nx.draw_networkx_edges(G, pos, ax=ax, edge_color='gray',
                        width=2, alpha=0.6, arrows=True,
                        arrowsize=20, arrowstyle='->')

# Draw nodes
node_sizes = [3000 if G.degree(node) > 2 else 2000 for node in G.nodes()]
nx.draw_networkx_nodes(G, pos, ax=ax, node_color=color,
                        node_size=node_sizes, alpha=0.9,
                        edgecolors='white', linewidths=2)

# Draw labels
if show_labels:
    labels = {node: node.split('-')[-1] if '-' in str(node) else str(node)[:8]
              for node in G.nodes()}
    nx.draw_networkx_labels(G, pos, labels, ax=ax, font_size=9,
                            font_weight='bold', font_color='white')

# Add statistics
stats_text = f'Nodes: {G.number_of_nodes()}\nLinks: {G.number_of_edges()}'
ax.text(0.02, 0.98, stats_text, transform=ax.transAxes,
        fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))
ax.set_title(title, fontsize=14, fontweight='bold', pad=20)
ax.axis('off')

```

User Interface (dashboard.py - excerpt)

```

# NETWORK TOPOLOGY GRAPH

# -----

st.header("🌐 Network Topology")

net = Network(height="600px", width="100%", directed=False)

for node in arp_nodes:
    label = node["ip_address"]
    title = f"""
    IP: {node.get('ip_address','')}
    MAC: {node.get('mac_address','')}
    Vendor: {node.get('vendor','Unknown')}
    Device: {node.get('device_type','Unknown')}
    """
    net.add_node(label, label=label, title=title)

for link in arp_links:
    if link["source"] in [n["ip_address"] for n in arp_nodes] \
        and link["target"] in [n["ip_address"] for n in arp_nodes]:
        net.add_edge(link["source"], link["target"])

net.save_graph("topology.html")

HtmlFile = open("topology.html", "r", encoding="utf-8")
st.components.v1.html(HtmlFile.read(), height=650)

```


B. PUBLICATION ACCEPTANCE PROOF

Gmail

Vedant Pandey <ved2003ant@gmail.com>

Full paper acceptance and Invitation to DIGI-SUSTAIN 2026 Conference

2 messages

ABV-IIITM Gwalior, DIGI-SUSTAIN (Conference Secretary) <confsect@iitm.ac.in>

To: Vedant Pandey <ved2003ant@gmail.com>

Mon, May 4, 2026 at 3:00 PM

Dear Author,

Greetings from the Department of Management Studies, Atal Bihari Vajpayee-Indian Institute of Information Technology and Management (ABV-IIITM), Gwalior.

We are pleased to inform you that your paper titled "Network Topology Discovery as a Digital Transformation Enabler: A Comparative Study of API, Protocol, and Scanning Approaches for Sustainable Enterprise Infrastructure" has been **accepted for presentation** at the International Conference on Digital Transformative Practices for a Sustainable Future (DIGI-SUSTAIN 2026), scheduled from 13th to 15th May 2026 in hybrid mode at ABV-IIITM Gwalior.

Your paper has been reviewed by the conference review committee and based on its academic merit and relevance to the conference theme; it has been recommended for inclusion in the conference presentation track.

We sincerely congratulate you on this achievement and appreciate your valuable contribution to the conference.

To confirm your participation and secure your presentation slot, kindly complete your registration at the earliest using the link below:
<https://forms.gle/yN6YzgAvJeKV2aM3A>

The payment QR code is attached for your convenience.

Registration Fee Structure:

- Rs. 5000 – On-campus presentation with consideration for journal publication
- Rs. 3000 – On-campus presentation
- Rs. 2000 – Online presentation

We encourage you to consider attending the conference at the ABV-IIITM Gwalior campus, as it offers enriched opportunities for academic networking, direct engagement with experts, and active participation in conference sessions and associated events.

Your accepted full paper will be considered for:

- Conference Proceedings
- Book of Abstracts
- Collaborating Journals (subject to peer review and editorial policies)

In addition to technical sessions, DIGI-SUSTAIN 2026 will feature several academic engagements, including:

C. PLAGIARISM REPORT

Venkatesh K

paper5678

Students Projects

NWC

SRM Institute of Science & Technology

Document Details

Submission ID
trn:old::1:3555784788

Submission Date
Apr 30, 2026, 3:00 PM GMT+5:30

Download Date
Apr 30, 2026, 3:05 PM GMT+5:30

File Name
Network_Topology_Discovery_Project_Report.pdf

File Size
1.8 MB

53 Pages

7,737 Words

41,681 Characters



Page 2 of 56 - Integrity Overview

Submission ID trn:old::1:3555784788

4% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- ▶ Bibliography
- ▶ Quoted Text

Match Groups

-  **18 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
-  **2 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 2%  Publications
- 3%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

PLAGIARISM REPORT

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY (Deemed to be University u/s 3 of UGC Act, 1956)		
Office of Controller of Examinations		
REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION/PROJECT REPORTS FOR UG/PG PROGRAMMES (To be attached in the dissertation/ project report)		
1	Name of the Candidate (IN BLOCK LETTERS)	VEDANT PANDEY CHIRANJEEV KUMAR
2	Address of the Candidate	Abode Valley, Potheri, 603203 S.I.S Safaa, Guduvancheri 603202
3	Registration Number	RA2211029010013 RA2211029010019
4	Date of Birth	05/06/2003 27/09/2004
5	Department	Networking and Communications
6	Faculty	Engineering and Technology, School of Computing
7	Title of the Dissertation/Project	Automated Network Topology Discovery And Analysis Using Cisco DNA Center
8	Whether the above project /dissertation is done by	Individual or group : (Strike whichever is not applicable) a) If the project/ dissertation is done in group, then how many students together completed the project : 2 b) Mention the Name & Register number of other candidates : VEDANT PANDEY [RA2211029010013] CHIRANJEEV KUMAR [RA2211029010019]
9	Name and address of the Supervisor / Guide	Dr. Rajaram. V Mail ID: rajaramv@srmist.edu.in Mobile Number: +91 98404 31173
10	Name and address of Co-Supervisor / Co- Guide (if any)	Dr. Venkatesh K Mail ID: venkatek2@srmist.edu.in Mobile Number: +91 99620 08082

11	Software Used	TurnItIn		
12	Date of Verification	30/04/2026		
13	Plagiarism Details: (to attach the final report from the software)			
Chapter	Title of the Chapter	Percentage of similarity index (including self citation)	Percentage of similarity index (Excluding self-citation)	% of plagiarism after excluding Quotes, Bibliography, etc.,
1	Introduction	1.0	1.0	1.0
2	Sprint Planning and Execution	0.3	0.3	0.3
3	Product Release Plan & Backlog	0.4	0.4	0.4
4	Cover Page & Index	1.0	1.0	1.0
5	Functional Test Cases	0.3	0.3	0.3
6	System Architecture	0.15	0.15	0.15
7	UI Designs	0.25	0.25	0.25
8	Sample Coding	0.1	0.1	0.1
9	Results and Discussion	0.2	0.2	0.2
10	Conclusion & Future Enhancements	0.2	0.2	0.2
Appendices		0.1	0.1	0.1
I / We declare that the above information have been verified and found true to the best of my / our knowledge.				
Signature of the Candidate		Name & Signature of the Staff (Who uses the plagiarism check software)		
Name & Signature of the Supervisor/ Guide		Name & Signature of the Co-Supervisor/Co-Guide		
Name & Signature of the HOD				